Swami Keshvanand Institute of Technology Management & Gramothan, Jaipur

Lab Manual

CSUP120: C Programming Lab

Semester: I

Session 2024-25

Faculty:

Ms. Ritikaa Sharma,
Assistant Professor

Mrs.Garima Garg,
Assistant Professor

Department of Information Technology

Swami Keshvanand Institute of Technology Management &

Edit with WPS Office

Gramothan, Ramnagaria, Jaipur-302017


Edit with WPS Office

Lab Manual
CSUP120: C Programming Lab

SESSION 2024-25

	AUTHOR/ OWNER	REVIEWED BY	APPROVED BY
NAME	Ms. Ritikaa Sharma Mrs. Garima Garg		Prof (Dr.) Mehul Mahrishi
DESIGNATION	Assistant Professor		HOD(IT)
SIGNATURE			

Edit with WPS Office

LAB ETHICS

Responsibilities of Users

Users are expected to follow some fairly obvious rules of conduct:



Always:

- Enter the lab on time and leave at proper time.
- Wait for the previous class to leave before the next class enters.
- Keep the bag outside in the respective racks.
- Utilize lab hours in the corresponding.
- Turn off the machine before leaving the lab unless a member of lab staff has specifically told you not to do so.
- Leave the labs at least as nice as you found them.
- If you notice a problem with a piece of equipment (e.g. a computer doesn't respond) or the room in general (e.g. cooling, heating, lighting) please report it to lab staff immediately. Do not attempt to fix the problem yourself.

Never :



- Don't abuse the equipment.
- Do not adjust the heat or air conditioners. If you feel the temperature is not properly set, inform lab staff; we will attempt to maintain a balance that is healthy for people and machines.
- Do not attempt to reboot a computer. Report problems to lab staff.

Edit with WPS Office

 Do not remove or modify any software or file without permission.


Edit with WPS Office

- Do not remove printers and machines from the network without being explicitly told to do so by lab staff.
- Don't monopolize equipment. If you're going to be away from your machine for more than 10 or 15 minutes, log out before leaving. This is both for the security of your account, and to ensure that others are able to use the lab resources while you are not.
- Don't use internet, internet chat of any kind in your regular lab schedule.
- Do not download or upload of MP3, JPG or MPEG files.
- No games are allowed in the lab sessions.
- No hardware including USB drives can be connected or disconnected in the labs without prior permission of the lab in-charge.
- No food or drink is allowed in the lab or near any of the equipment. Aside from the fact that it leaves a mess and attracts pests, spilling anything on a keyboard or other piece of computer equipment could cause permanent, irreparable, and costly damage. (and in fact *has*) If you need to eat or drink, take a break and do so in the canteen.
- Don't bring any external material in the lab, except your lab record, copy and books.
- Don't bring the mobile phones in the lab. If necessary then keep them in silence mode.
- Please be considerate of those around you, especially in terms of noise level. While labs are a natural place for conversations of all types, kindly keep the volume turned down.

Note: If you are having problems or questions, please go to either the faculty, lab in-charge or the lab supporting staff. They will help you. We need your full support and cooperation for smooth functioning of the lab.

Edit with WPS Office

INSTRUCTIONS

Before Entering in lab

- All the students are supposed to prepare the theory regarding the next experiment/ Program.
- Students are supposed to bring their lab records as per their lab schedule.
- Previous experiment/program should be written in the lab record.
- If applicable trace paper/graph paper must be pasted in lab record with proper labeling.
- All the students must follow the instructions, failing which he/she may not be allowed in the lab.

While working in lab

- Adhere to experimental schedule as instructed by the lab in-charge/faculty.
- Get the previously performed experiment/ program signed by the faculty/ lab in charge.
- Get the output of current experiment/program checked by the faculty/ lab in charge in the lab copy.
- Each student should work on his/her assigned computer at each turn of the lab.
- Take responsibility of valuable accessories.
- Concentrate on the assigned practical and don't play games.
- If anyone is caught red-handed carrying any equipment of the lab, then he/she will have to face serious consequences.

Edit with WPS Office

Marking/Assessment System

Total Marks -100

Distribution of Marks - 60 (Sessional)

Attendance	File Work	Performance	Viva	Total
10	10	20	20	60

Distribution of Marks - 40 (End Term)

File Work	Performance	Viva	Total
10	20	10	40

Edit with WPS Office

Rajasthan Technical University, Kota

Syllabus

I-year I Semester B.Tech. Information Technology

CSUP120: C Programming Lab

Credit: 1 Max. Marks: 100 (IA: 60, ETE:40)

L + 0T + 3P End Term Exam: 3 Hours

Edit with WPS Office

LAB PLAN

Total number of experiments 10

Total number of turns required 10

Number of turns required for

Experiment Number	Turns	Scheduled Day
Exp. 1	1	Day 1
Exp. 2	1	Day 2
Exp. 3	1	Day 3
Exp. 4	1	Day 4
Exp. 5	1	Day 5
Exp. 6	1	Day 6
Exp. 7	1	Day 7
Exp. 8	1	Day 8
Exp. 9	1	Day 9
Exp. 10	1	Day 10

Edit with WPS Office

Prerequisites for a C Programming Lab

Before starting a C programming lab, students should have basic knowledge of:

1. Mathematical and Logical Skills:
    - Understanding of basic arithmetic operations.
    - Basic problem-solving skills.
2. Basic Computer Knowledge:
    - Familiarity with operating systems (e.g., Windows, Linux) and using the command line.
    - Understanding of file creation, saving, and executing programs.
3. Fundamentals of Programming:
    - Concepts of variables, data types, and simple algorithms.
    - Logical thinking and the ability to follow program flow.
4. Introduction to C Programming:
    - Knowledge of the basic structure of a C program.
    - Understanding how to compile and run C code using an Integrated Development Environment (IDE) or a compiler.

Learning Objectives for C Lab

Upon completing the C programming lab, students should be able to:

1. Understand and Use Data Types:
    - Understand different data types like integers, floats, characters, and pointers.
    - Use arrays and structures effectively.
2. Apply Control Structures:
    - Implement decision-making constructs like if-else, switch-case.
    - Use loops (for, while, do-while) to iterate through operations.
3. Write and Use Functions:
    - Understand the concept of functions, including declaration, definition, and usage.
    - Apply function calling and recursion.
4. Handle Input/Output Operations:
    - Use printf and scanf for formatted input/output.
    - Understand file I/O operations (opening, reading, writing, and closing files).
5. Understand Memory Management:
    - Work with pointers and dynamic memory allocation (e.g., malloc, calloc, free).
6. Implement Searching and Sorting Algorithms:
    - Understand and implement basic algorithms like linear search, binary search, and sorting techniques.
7. Debug and Troubleshoot Programs:
    - Identify syntax and logical errors in a program and debug them effectively.
    - Use development tools like debuggers to trace program execution.

Software Required for C Lab


Edit with WPS Office

1. C Compiler:
    - GCC (GNU Compiler Collection): A popular open-source C compiler available on Linux, Windows (via MinGW), and macOS.
    - Turbo C/C++: An older IDE still used in some educational settings.
    - Clang: Another open-source compiler, which is known for faster compilation.
2. Integrated Development Environments (IDEs):
    - Code::Blocks: A widely used open-source IDE for C and C++.
    - Dev-C++: A simple IDE with GCC included.
    - Visual Studio Code (VS Code): A modern code editor with extensions to support C development.
    - Eclipse CDT: A popular IDE for C/C++ with debugging and compiling features.
3. Debugger Tools:
    - GDB (GNU Debugger): A powerful debugger for C programs, often bundled with GCC.
    - IDE-based Debuggers: Code::Blocks, Eclipse, and VS Code have built-in debuggers.
4. Text Editor (if using command line):
    - Notepad++ (Windows).
    - Vim or nano (Linux/Unix-based systems).
    - Sublime Text (Cross-platform).

Hardware Requirements for C Lab

1. Basic Hardware:
    - Processor: At least a dual-core processor (e.g., Intel Core i3 or equivalent).
    - RAM: Minimum of 2 GB (4 GB recommended for smooth multitasking).
    - Hard Disk: At least 20 GB of free space.
2. Operating System:
    - Windows, Linux, or macOS (Linux is often preferred for its native support of C development tools).
3. Other Peripherals:
    - Monitor: Basic monitor (at least 1366x768 resolution).
    - Keyboard & Mouse: Essential for typing code and navigation.
    - Power Supply: A reliable power source, especially for desktop computers in a lab setting.

Lab Assignment -1 (Basic C Programs)

Structure of program, Learn about C library, Preprocessor directives, input-output


Edit with WPS Office

statements, Data Types and tokens in C programming

C is a general-purpose, procedural, high-level programming language used in the development of computer software and applications, system programming, games, and more.This is known for its simplicity and efficiency. It is the best choice to start with programming as it gives you a foundational understanding of programming.

Structure of the C program

1. By structure, it is meant that any program can be written in this structure only. Writing a C program in any other structure will lead to a Compilation Error. The Components of a C Program are as follows:

1. Header Files Inclusion – [#include <stdio.h>]

The first and foremost component is the inclusion of the Header files in a C program. A header file is a file with extension .h which contains C function declarations and macro definitions to be shared between several source files. All lines that start with # are processed by a preprocessor which is a program invoked by the compiler. In the above example, the preprocessor copies the preprocessed code of stdio.h to our file. The .h files are called header files in C.
Some of the C Header files:

- stdio.h – Defines core input and output functions
- stdlib.h – Defines numeric conversion functions, pseudo-random number generator, and memory allocation
- string.h – Defines string handling functions
- math.h – Defines common mathematical functions.
- stddef.h – Defines several useful types and macros.
- stdint.h – Defines exact width integer types.

2. Main Method Declaration – [void main()]

The next part of a C program is to declare the main() function. It is the entry point of a C program and the execution typically begins with the first line of the main(). The empty brackets indicate that the main doesn't take any parameter The int that was written before the main indicates the return type of main().

3. Body of Main Method – Line 3 to Line 6 [enclosed in {}]

The body of a function in the C program refers to statements that are a part of that function. It can be anything like manipulations, searching, sorting, printing, etc. A pair of curly brackets define the body of a function. All functions must start and end with curly brackets.

4. Statement – printf("Hello World");


Edit with WPS Office

Statements are the instructions given to the compiler. In C, a statement is always terminated by a **semicolon (;).** In this particular case, we use printf() function to instruct the compiler to display "Hello World" text on the screen.

5. getch() Statement – [getch();]

The last part of any C function is the getch() statement. This is a function which holds the console until a key is pressed.

Comment in C

The **comments in C** are human-readable explanations or notes in the source code of a C program. A comment makes the program easier to read and understand. These are the statements that are not executed by the compiler or an interpreter.

Types of comments in C: In C there are two types of comments in C language:

- **Single-line comment**
- **Multi-line comment**

Syntax of Single Line C Comment

```
// This is a single line comment
```

Syntax of Multi-Line C Comment

```
/*Comment starts
   continues
   continues
   .
   .
   .
Comment ends*/
```

Token in C

A token in C can be defined as the smallest individual element of the C programming language that is meaningful to the compiler. It is the basic component of a C program.

Types of Tokens in C
The tokens of C language can be classified into six types based on the functions they are used to perform. The types of C tokens are as follows:

1. **Keywords**
2. **Identifiers**
3. **Constants**

Edit with WPS Office

4. **Strings**
5. **Special Symbols**
6. **Operators**

Keywords

The keywords are pre-defined or reserved words in a programming language. Each keyword is meant to perform a specific function in a program. Since keywords are referred names for a compiler, they can't be used as variable names because by doing so, we are trying to assign a new meaning to the keyword which is not allowed. You cannot redefine keywords. However, you can specify the text to be substituted for keywords before compilation by using C preprocessor directives. **C** language supports **32** keywords which are given below:

auto	**double**	**int**	**struct**
break	**else**	**long**	**switch**
case	**enum**	**register**	**typedef**
char	**extern**	**return**	**union**
const	**float**	**short**	**unsigned**
continue	**for**	**signed**	**void**
default	**goto**	**sizeof**	**volatile**
do	**if**	**static**	**while**

Identifiers

Identifiers are used as the general terminology for the naming of variables, functions, and arrays. These are user-defined names consisting of an arbitrarily long sequence of letters and digits with either a letter or the underscore(_) as a first character. Identifier names must differ in spelling and case from any keywords. You cannot use keywords as identifiers; they are reserved for special use. Once declared, you can use the identifier in later program statements to refer to the associated value. A special identifier called a statement label can be used in goto statements.

Rules for Naming Identifiers

Certain rules should be followed while naming c identifiers which are as follows:

- They must begin with a letter or underscore(_).
- They must consist of only letters, digits, or underscore. No other special character is allowed.
- It should not be a keyword.
- It must not contain white space.
- It should be up to 31 characters long as only the first 31 characters are significant.

For example,

- **main:** method name.
- **a:** variable name.

Constants

Edit with WPS Office

The constants refer to the variables with fixed values. They are like normal variables but with the difference that their values can not be modified in the program once they are defined.

Constants may belong to any of the data types.

Examples of Constants in C

```
const int c_var = 20;
const int* const ptr = &c_var;
```

Strings

Strings are nothing but an array of characters ended with a null character ('\0'). This null character indicates the end of the string. Strings are always enclosed in double quotes. Whereas, a character is enclosed in single quotes in C and C++.

Examples of String

```
char string[20] = {'g', 'e', 'e', 'k', 's', 'f', 'o', 'r', 'g', 'e', 'e', 'k', 's', '\0'};
char string[20] = "geeksforgeeks";
char string [] = "geeksforgeeks";
```

Special Symbols

The following special symbols are used in C having some special meaning and thus, cannot be used for some other purpose. Some of these are listed below:

- **Brackets[]:** Opening and closing brackets are used as array element references. These indicate single and multidimensional subscripts.
- **Parentheses():** These special symbols are used to indicate function calls and function parameters.
- **Braces{}:** These opening and ending curly braces mark the start and end of a block of code containing more than one executable statement.
- **Comma (,):** It is used to separate more than one statement like for separating parameters in function calls.
- **Colon(:):** It is an operator that essentially invokes something called an initialization list.
- **Semicolon(;):** It is known as a statement terminator. It indicates the end of one logical entity. That's why each individual statement must be ended with a semicolon.
- **Asterisk (*):** It is used to create a pointer variable and for the multiplication of variables.
- **Assignment operator(=):** It is used to assign values and for logical operation validation.
- **Pre-processor (#):** The preprocessor is a macro processor that is used automatically by the compiler to transform your program before actual compilation.
- **Period (.):** Used to access members of a structure or union.
- **Tilde(~):** Bitwise One's Complement Operator.

Operators


Edit with WPS Office

Operators are symbols that trigger an action when applied to C variables and other objects. The data items on which operators act are called operands.
Depending on the number of operands that an operator can act upon, operators can be classified as follows:

- **Unary Operators:** Those operators that require only a single operand to act upon are known as unary operators.For Example increment and decrement operators
- **Binary Operators:** Those operators that require two operands to act upon are called binary operators. Binary operators can further are classified into:

1. Arithmetic operators
2. Relational Operators
3. Logical Operators
4. Assignment Operators
5. Bitwise Operator

- **Ternary Operator**: The operator that requires three operands to act upon is called the ternary operator. Conditional Operator(?) is also called the ternary operator.

Basic Data Types

The data type specifies the size and type of information the variable will store.

Data Type	Description	Example
int	Stores whole numbers, without decimals	1
float	Stores fractional numbers, containing one or more decimals. Sufficient for storing 6-7 decimal digits	1.99
double	Stores fractional numbers, containing one or more decimals.	1.99

Edit with WPS Office

	Sufficient for storing 15 decimal digits	
char	Stores a single character/letter/number, or ASCII values	'A'

Basic Format Specifiers

There are different format specifiers for each data type. Here are some of them:

Format Specifier	Data Type
%d	int
%f	float
%lf	double
%c	char

1. **Write a program to display “Hello ” message**

```
#include <stdio.h>  // Required for input-output functions

void main() {
    // Display "Hello" message
    clrscr();
    printf("Hello");
    getch();
}
```

Output

Edit with WPS Office

Hello

2. **Write a program to add any two numbers.**

```
#include <stdio.h>  // Required for input-output functions

void main() {
    int num1, num2, sum;

    // Input two numbers
    printf("Enter first number: ");
    scanf("%d", &num1);

    printf("Enter second number: ");
    scanf("%d", &num2);

    // Calculate the sum
    sum = num1 + num2;

    // Display the result
    printf("The sum of %d and %d is %d", num1, num2, sum);
}
```

Output

```
Enter first number: 5
Enter second number: 10
The sum of 5 and 10 is 15
```

3. **Write a program to convert temperature from Celsius to Fahrenheit.**

```
#include <stdio.h>  // Required for input-output functions

void main() {
    float celsius, fahrenheit;

    // Input temperature in Celsius
    printf("Enter temperature in Celsius: ");
    scanf("%f", &celsius);

    // Convert Celsius to Fahrenheit
    fahrenheit = (celsius * 9 / 5) + 32;

    // Display the result
```


Edit with WPS Office

```
    printf("Temperature in Fahrenheit: %.2f", fahrenheit);
}
```

Output

```
Enter temperature in Celsius: 0
Temperature in Fahrenheit: 32.00
```

4. **Write a program to find the area and circumference of a circle.**

```
#include <stdio.h>  // Required for input-output functions
#define PI 3.14159   // Define constant for Pi

void main() {
    float radius, area, circumference;

    // Input radius of the circle
    printf("Enter the radius of the circle: ");
    scanf("%f", &radius);

    // Calculate area and circumference
    area = PI * radius * radius;
    circumference = 2 * PI * radius;

    // Display the results
    printf("Area of the circle: %.2f\n", area);
    printf("Circumference of the circle: %.2f\n

}
```

Output:

```
Enter the radius of the circle: 5
Area of the circle: 78.54
Circumference of the circle: 31.42
```


Edit with WPS Office

Lab Assignment-2(Types of Operators)

Different types of operator Arithmetic, Relational, Logical,Increment/Decrement, Conditional Operators

In C programming, operators are symbols used to perform operations on variables and values. They are fundamental to manipulating data in a C program. Operators in C can be classified into several types:

1. Arithmetic Operators

Arithmetic operators are used to perform basic mathematical operations such as addition, subtraction, multiplication, division, etc.

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus (Remainder)

Edit with WPS Office

Operator	Description

2. Relational (Comparison) Operators

Relational operators are used to compare two values or expressions. The result of these operations is either true (1) or false (0).

Operator	Description
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to

Edit with WPS Office

Operator	Description
<=	Less than or equal to

3. Logical Operators

Logical operators are used to combine or invert the results of relational operators. They are typically used in conditions.

Operator	Description	Example
&&	Logical AND	a && b
`		`
!	Logical NOT	!a

4. Assignment Operators

Assignment operators are used to assign values to variables. The basic assignment operator is =, but there are compound assignment operators that combine other operations with assignment.

Operator	Description	Exa
=	Assigns value	a = b
+=	Add and assign	a += b

Edit with WPS Office

Operator	Description	Exa
-=	Subtract and assign	a -= b
*=	Multiply and assign	a *= b
/=	Divide and assign	a /= b
%=	Modulus and assign	a %= b

5. Bitwise Operators

Bitwise operators perform operations on individual bits of integer data types. They are useful in low-level programming tasks such as system programming and device drivers.

Operator	Description	Example
&	Bitwise AND	a & b
`	`	Bitwise OR
^	Bitwise XOR	a ^ b
~	Bitwise NOT (One's Complement)	~a
<<	Left shift	a << 2
>>	Right shift	a >> 2

6. Increment and Decrement Operators

These are used to increase or decrease the value of a variable by 1.



Edit with WPS Office

Operator	Description	Example
++	Increment by 1	a++ or ++a
--	Decrement by 1	a-- or --a

Pre-Increment (++a): Increments the value before it is used.
Post-Increment (a++): Increments the value after it is used.

7. Conditional (Ternary) Operator

The conditional operator (? :) is a shorthand way to write an if-else statement. It evaluates a condition and returns one of two values based on whether the condition is true or false.

Operator	Description	Example
? :	Conditional expression	condition ? expr1 : expr2

8. Special Operators

Some special operators in C include:

Comma Operator (,): Used to separate multiple expressions.
Sizeof Operator (sizeof**)**: Returns the size of a data type or variable in bytes.
Pointer Operators (& **and** ***)**: Used for working with memory addresses.

5. Write a program to demonstrate the working of Arithmetic operators.
   #include <stdio.h> // Required for input-output functions

Edit with WPS Office

```
void main() {
    int num1, num2;
    int sum, difference, product, quotient, remainder;

    // Input two numbers
    printf("Enter first number: ");
    scanf("%d", &num1);

    printf("Enter second number: ");
    scanf("%d", &num2);

    // Arithmetic operations
    sum = num1 + num2;              // Addition
    difference = num1 - num2;       // Subtraction
    product = num1 * num2;          // Multiplication
    quotient = num1 / num2;         // Division
    remainder = num1 % num2;        // Modulus (Remainder)

    // Display the results
    printf("Sum: %d\n", sum);
    printf("Difference: %d\n", difference);
    printf("Product: %d\n", product);
    printf("Quotient: %d\n", quotient);
    printf("Remainder: %d\n", remainder);
}
```

Output

```
Enter first number: 15
Enter second number: 4

Sum: 19
Difference: 11
Product: 60
Quotient: 3
Remainder: 3
```

6. **Write a program to demonstrate the working of Logical operators.**

```
#include <stdio.h>  // Required for input-output functions

void main() {
    int a, b;
    int and_result, or_result, not_result;

    // Input two integers
```


Edit with WPS Office

```
    printf("Enter first number (a): ");
    scanf("%d", &a);

    printf("Enter second number (b): ");
    scanf("%d", &b);

    // Logical AND (&&): True if both conditions are true
    and_result = (a > 0) && (b > 0);

    // Logical OR (||): True if at least one condition is true
    or_result = (a > 0) || (b > 0);

    // Logical NOT (!): Reverses the result (True becomes False and vice versa)
    not_result = !(a > 0);

    // Display the results
    printf("\nLogical AND (a > 0 && b > 0): %d", and_result);  // 1 if true, 0 if false
    printf("\nLogical OR (a > 0 || b > 0): %d", or_result);     // 1 if true, 0 if false
    printf("\nLogical NOT (!(a > 0)): %d", not_result);         // 1 if true, 0 if false
}
```

Output

```
Enter first number (a): -2
Enter second number (b): -1

Logical AND (a > 0 && b > 0): 0
Logical OR (a > 0 || b > 0): 0
Logical NOT (!(a > 0)): 1
```

7. **Write a program to demonstrate the working of Increment / Decrement operators.**

```
#include <stdio.h>  // Required for input-output functions

void main() {
    int x, y;

    // Input a number
    printf("Enter a number (x): ");
    scanf("%d", &x);

    // Post-increment (x++)
    y = x++;  // x is assigned to y first, then incremented
    printf("After post-increment (y = x++): x = %d, y = %d\n", x, y);
```

Edit with WPS Office

```
    // Pre-increment (++x)
    y = ++x;  // x is incremented first, then assigned to y
    printf("After pre-increment (y = ++x): x = %d, y = %d\n", x, y);

    // Post-decrement (x--)
    y = x--;  // x is assigned to y first, then decremented
    printf("After post-decrement (y = x--): x = %d, y = %d\n", x, y);

    // Pre-decrement (--x)
    y = --x;  // x is decremented first, then assigned to y
    printf("After pre-decrement (y = --x): x = %d, y = %d\n", x, y);
}
```

Output:

```
Enter a number (x): 5

After post-increment (y = x++): x = 6, y = 5
After pre-increment (y = ++x): x = 7, y = 7
After post-decrement (y = x--): x = 6, y = 7
After pre-decrement (y = --x): x = 5, y = 5
```

8. **Write a program to find largest among three numbers using conditional/ternary operator**

```
#include <stdio.h>  // Required for input-output functions

void main() {
    int num1, num2, num3, largest;

    // Input three numbers
    printf("Enter first number: ");
    scanf("%d", &num1);

    printf("Enter second number: ");
    scanf("%d", &num2);

    printf("Enter third number: ");
    scanf("%d", &num3);

    // Using the ternary operator to find the largest number
    largest = (num1 > num2) ?
              ((num1 > num3) ? num1 : num3) :
              ((num2 > num3) ? num2 : num3);
```

Edit with WPS Office

```
    // Display the result
    printf("The largest number is: %d", largest);
}
```

Output

```
Enter first number: 50
Enter second number: 20
Enter third number: 30
The largest number is: 50
```

Lab Assignment-3(Conditional Statements)
Learn About Various Conditional statements of if-else, nested if-else,if-else-if ladder and Switch case statements

if-else **Statement**

The if-else statement allows you to execute one block of code if a condition is true and another block if the condition is false.

Syntax:

```
if (condition) {
    // Code to execute if condition is true
} else {
    // Code to execute if condition is false
}
```

Nested if-else **Statement**

A nested if-else statement means placing one if-else statement inside another. It helps to check multiple conditions within the same if block.

Syntax:

```
if (condition1) {
    if (condition2) {
        // Code to execute if both condition1 and condition2 are true
```

Edit with WPS Office

```
    } else {
        // Code to execute if condition1 is true but condition2 is false
    }
} else {
    // Code to execute if condition1 is false
}
```

if-else-if Ladder

The if-else-if ladder is used when you need to check multiple conditions. Once a true condition is found, the corresponding block of code is executed, and the rest of the conditions are skipped.

Syntax:

```
if (condition1) {
    // Code to execute if condition1 is true
} else if (condition2) {
    // Code to execute if condition2 is true
} else if (condition3) {
    // Code to execute if condition3 is true
} else {
    // Code to execute if none of the above conditions are true
}
```

switch Case Statement

The switch statement is used to evaluate a variable and execute the block of code corresponding to the matched case value. It is an alternative to using multiple if-else-if statements.

Syntax:

```
switch (expression) {
    case constant1:
        // Code to execute if expression == constant1
        break;

    case constant2:
        // Code to execute if expression == constant2
        break;

    // Additional cases...

    default:
        // Code to execute if none of the cases match
}
```

Edit with WPS Office

9. **Write a program to check whether a number is Odd or Even.**

```
#include <stdio.h>

int main() {
    int number;

    // Input a number from the user
    printf("Enter an integer: ");
    scanf("%d", &number);

    // Check if the number is even or odd
    if (number % 2 == 0) {
        printf("%d is an even number.\n", number);
    } else {
        printf("%d is an odd number.\n", number);
    }

    return 0;
}
```

```
Enter an integer: 5
5 is an odd number.

Enter an integer: 10
10 is an even number.
```

10. **Write a program to check a given character is Alphabet or Not.**

```
#include <stdio.h>

int main() {
 char ch;

// Input a character from the user
 printf("Enter a character: ");
  scanf(" %c", &ch);
```


Edit with WPS Office

```
    // Check if the character is an alphabet
    if ((ch >= 'A' && ch <= 'Z') || (ch >= 'a' && ch <= 'z')) {
            printf("%c is an alphabet.\n", ch);
    } else {
            printf("%c is not an alphabet.\n", ch);
   }

    return 0;
  }
```

Output:

```
Enter a character: G
G is an alphabet.

Enter a character: 5
5 is not an alphabet.

Enter a character: z
z is an alphabet.

Enter a character: *
* is not an alphabet.
```

11. **Write a program using switch case statement to display from Monday to Sunday**

```
#include <stdio.h>

int main() {
    int day;

    // Input a number from the user
    printf("Enter a number (1-7) to display the corresponding day of the week: ");
    scanf("%d", &day);

    // Use switch case to display the corresponding day
    switch (day) {
        case 1:
            printf("Monday\n");
            break;
        case 2:
            printf("Tuesday\n");
            break;
        case 3:
            printf("Wednesday\n");
```


Edit with WPS Office

```
            break;
        case 4:
            printf("Thursday\n");
            break;
        case 5:
            printf("Friday\n");
            break;
        case 6:
            printf("Saturday\n");
            break;
        case 7:
            printf("Sunday\n");
            break;
        default:
            printf("Invalid input! Please enter a number between 1 and 7.\n");
            break;
    }

    return 0;
}
```

Output:

```
Enter a number (1-7) to display the corresponding day of the week: 3
Wednesday

Enter a number (1-7) to display the corresponding day of the week: 7
Sunday

Enter a number (1-7) to display the corresponding day of the week: 8
Invalid input! Please enter a number between 1 and 7.
```

Lab Assignment-4(Looping Statements)
Learn about iterative statements like while, do-while, for loop, continue and break statements

In C programming, loops are used to execute a block of code repeatedly based on a condition. The while, do-while, and for loops are the three main types of loops. The continue and break statements provide more control over the flow of loops.

1. while **Loop**

Edit with WPS Office

The while loop executes a block of code as long as a specified condition is true. The condition is checked before the loop body is executed.

Syntax:

```
while (condition) {
    // Code to be executed as long as the condition is true
}
```

do-while Loop

The do-while loop is similar to the while loop, but the condition is checked **after** the loop body is executed. This ensures that the loop is executed at least once, even if the condition is false initially.

Syntax:

```
do {
    // Code to be executed
} while (condition);
```

for Loop

The for loop is used when the number of iterations is known beforehand. It consists of three parts: initialization, condition, and increment/decrement, all within the loop header.

Syntax:

```
for (initialization; condition; increment/decrement) {
    // Code to be executed
}
```

break Statement

The break statement is used to exit a loop (or a switch statement) prematurely. When a break is encountered inside a loop, the loop terminates, and control moves to the statement after the loop.

Syntax:

```
break;
```

continue Statement

The continue statement skips the current iteration of the loop and moves to the next iteration. The loop does not terminate; instead, it skips the remaining code for the current iteration and checks the loop condition again.

Syntax:

```
continue;
```


Edit with WPS Office

12. **Write a program to display the first 10 Natural numbers and display their sum.**

```
#include <stdio.h>  // Required for input-output functions

void main() {
    int i, sum = 0;

    // Display the first 10 natural numbers
    printf("The first 10 natural numbers are:\n");
    for(i = 1; i <= 10; i++) {
        printf("%d ", i);
        sum += i;  // Add each number to the sum
    }

    // Display the sum of the numbers
    printf("\nSum of the first 10 natural numbers is: %d", sum);
}
```

Output

```
The first 10 natural numbers are:
1 2 3 4 5 6 7 8 9 10
Sum of the first 10 natural numbers is: 55
```

13. **Write a program to print a Table of a number using a for loop.**

```
#include <stdio.h>  // Required for input-output functions

void main() {
    int num, i;

    // Input a number from the user
    printf("Enter a number to print its multiplication table: ");
    scanf("%d", &num);

    // Print the multiplication table using a for loop
    printf("Multiplication table of %d:\n", num);
    for(i = 1; i <= 10; i++) {
        printf("%d x %d = %d\n", num, i, num * i);
    }
}
```

Output


Edit with WPS Office

```
Enter a number to print its multiplication table: 7
Multiplication table of 7:
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

14. Write a program to find whether a number is Prime number or not.

```
#include <stdio.h>  // Required for input-output functions

void main() {
    int num, i, isPrime = 1;  // Assume the number is prime initially

    // Input a number from the user
    printf("Enter a number to check if it's a prime number: ");
    scanf("%d", &num);

    // Check if number is less than 2 (not prime)
    if (num < 2) {
        isPrime = 0;  // Numbers less than 2 are not prime
    } else {
        // Check if the number has any divisor other than 1 and itself
        for(i = 2; i <= num / 2; i++) {
            if(num % i == 0) {
                isPrime = 0;  // Number is divisible by i, so it's not prime
                break;
            }
        }
    }

    // Display result
    if (isPrime == 1)
        printf("%d is a prime number.", num);
    else
        printf("%d is not a prime number.", num);
}
```

Output

Edit with WPS Office

Enter a number to check if it's a prime number: 7
7 is a prime number.

15. **Write a program to find Reverse of a given number.**

```
#include <stdio.h>  // Required for input-output functions

void main() {
    int num, reverse = 0, remainder;

    // Input a number from the user
    printf("Enter a number to find its reverse: ");
    scanf("%d", &num);

    // Reverse the number using a loop
    while (num != 0) {
        remainder = num % 10;         // Get the last digit of the number
        reverse = reverse * 10 + remainder;  // Build the reverse number
        num = num / 10;               // Remove the last digit
    }

    // Display the reversed number
    printf("The reverse of the given number is: %d", reverse);
}
```

Output
Enter a number to find its reverse: 9876
The reverse of the given number is: 6789

16. **Write a program to find Factorial of given number**

```
#include <stdio.h>  // Required for input-output functions

void main() {
    int num, i;
    unsigned long long factorial = 1;  // Use a large type to handle bigger factorials

    // Input a number from the user
    printf("Enter a number to find its factorial: ");
    scanf("%d", &num);

    // Check if the number is negative
    if (num < 0) {
        printf("Factorial of a negative number doesn't exist.");
    } else {
```


Edit with WPS Office

```
        // Calculate the factorial using a for loop
        for(i = 1; i <= num; i++) {
            factorial *= i;  // Multiply factorial by the current number
        }
        // Display the result
        printf("Factorial of %d is: %llu", num, factorial);
    }
}
```

Enter a number to find its factorial: 0
Factorial of 0 is: 1

Lab Assignment-5 (Pattern based problem using Nested loop)
Nested for loop, nested while loop, different kind of patterns.

Nested for Loop

A nested for loop is a for loop inside another for loop. This allows you to iterate over a two-dimensional structure.

Syntax:

```
for (initialization1; condition1; increment1) {
    for (initialization2; condition2; increment2) {
        // Code to execute in the inner loop
    }
    // Code to execute after the inner loop
}
```

Nested while Loop

A nested while loop is a while loop inside another while loop. The same logic applies here as with nested for loops.

Syntax:

```
while (condition1) {
    while (condition2) {
        // Code to execute in the inner loop
    }
    // Code to execute after the inner loop
}
```

Edit with WPS Office

17. **Write a program to print half pyramid using * symbol.**

```
#include <stdio.h>  // Required for input-output functions

void main() {
    int rows, i, j;

    // Input the number of rows for the pyramid
    printf("Enter the number of rows: ");
    scanf("%d", &rows);

    // Loop to print the half pyramid
    for(i = 1; i <= rows; i++) {
        for(j = 1; j <= i; j++) {
            printf("* ");  // Print the '*' symbol
        }
        printf("\n");  // Move to the next line after each row
    }
}
```

```
Enter the number of rows: 5

*
* *
* * *
* * * *
* * * * *
```

18. **Write a program to print half pyramid using numbers.**

```
#include <stdio.h>  // Required for input-output functions

void main() {
    int rows, i, j;

    // Input the number of rows for the pyramid
    printf("Enter the number of rows: ");
    scanf("%d", &rows);

    // Loop to print the half pyramid
    for(i = 1; i <= rows; i++) {
        for(j = 1; j <= i; j++) {
            printf("%d ", j);  // Print the current number (j)
```


Edit with WPS Office

```
        }
        printf("\n");  // Move to the next line after each row
    }
}
```

```
Enter the number of rows: 5

1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

19. Write a program to print full pyramid using * symbol.

```
#include <stdio.h>  // Required for input-output functions

void main() {
    int rows, i, j, space;

    // Input the number of rows for the pyramid
    printf("Enter the number of rows: ");
    scanf("%d", &rows);

    // Loop to print the full pyramid
    for(i = 1; i <= rows; i++) {
        // Print spaces for pyramid alignment
        for(space = 1; space <= rows - i; space++) {
            printf("  ");  // Two spaces for proper alignment
        }

        // Print '*' symbols for each row
        for(j = 1; j <= 2 * i - 1; j++) {
            printf("* ");
        }

        // Move to the next line after each row
        printf("\n");
    }
}
```

```
Enter the number of rows: 5

        *
      * * *
```

Edit with WPS Office

```
   * * * * *
  * * * * * * *
* * * * * * * * *
```

Lab Assignment-6 (Array)
Learn about array , types of array- one dimensional and two dimensional(matrix) operation on array like searching , sorting

An **array** is a collection of elements of the same data type stored in contiguous memory locations. It allows you to store multiple values of the same type under a single variable name.

Types of Arrays

1. **One-Dimensional Array**
2. **Two-Dimensional Array (Matrix)**

One-Dimensional Array


Edit with WPS Office

A one-dimensional array is a simple linear array, where elements are stored in a single row. The elements are accessed by a single index.

Syntax:

data_type array_name[size];

Two-Dimensional Array (Matrix)

A two-dimensional array (or matrix) is an array of arrays. It can be visualized as a table with rows and columns. It is used to represent data in a grid-like format.

Syntax:

data_type array_name[rows][columns];

Arrays in C allow various operations like **searching** for elements, **sorting** elements, etc. Let's explore these operations.

---

1. Searching an Element in an Array

Linear Search

Linear search is a simple searching algorithm where each element is compared to the target value until it is found.

Binary Search

Binary search works on sorted arrays and reduces the search space by half after each comparison.

Sorting an Array

Sorting involves arranging the elements in a specified order, typically ascending or descending. Common sorting algorithms include **Bubble Sort**, **Selection Sort**, and **Insertion Sort**.

20. Write a program to calculate Sum and Average of 10 elements of given Array.

```
#include <stdio.h>  // Required for input-output functions

void main() {
    int arr[10], i;
    int sum = 0;
    float average;

    // Input 10 elements into the array
```


Edit with WPS Office

```
    printf("Enter 10 elements for the array:\n");
    for(i = 0; i < 10; i++) {
        printf("Element %d: ", i + 1);
        scanf("%d", &arr[i]);
        sum += arr[i];  // Add each element to sum
    }

    // Calculate the average
    average = sum / 10.0;

    // Display the sum and average
    printf("Sum of the array elements: %d\n", sum);
    printf("Average of the array elements: %.2f\n", average);
}
```

```
Enter 10 elements for the array:
Element 1: 5
Element 2: 10
Element 3: 15
Element 4: 20
Element 5: 25
Element 6: 30
Element 7: 35
Element 8: 40
Element 9: 45
Element 10: 50

Sum of the array elements: 275
Average of the array elements: 27.50
```

21. **Write a program to find the maximum no in the Array.**

```
#include <stdio.h>  // Required for input-output functions

void main() {
    int arr[10], i;
    int max;

    // Input 10 elements into the array
    printf("Enter 10 elements for the array:\n");
    for(i = 0; i < 10; i++) {
        printf("Element %d: ", i + 1);
        scanf("%d", &arr[i]);
    }

    // Assume the first element is the maximum
```


Edit with WPS Office

```
    max = arr[0];

    // Loop through the array to find the maximum element
    for(i = 1; i < 10; i++) {
        if(arr[i] > max) {
            max = arr[i];  // Update max if the current element is greater
        }
    }

    // Display the maximum element
    printf("The maximum number in the array is: %d\n", max);
}
```

```
Enter 10 elements for the array:
Element 1: 5
Element 2: 12
Element 3: 3
Element 4: 25
Element 5: 8
Element 6: 19
Element 7: 4
Element 8: 30
Element 9: 2
Element 10: 15

The maximum number in the array is: 30
```

22. Write a program for Addition of two matrices.

```
#include <stdio.h>  // Required for input-output functions

#define MAX 10  // Maximum size for the matrices

void main() {
    int a[MAX][MAX], b[MAX][MAX], sum[MAX][MAX];
    int rows, cols, i, j;

    // Input the number of rows and columns
    printf("Enter the number of rows and columns: ");
    scanf("%d %d", &rows, &cols);

    // Input elements of the first matrix
    printf("Enter elements of the first matrix:\n");
    for(i = 0; i < rows; i++) {
        for(j = 0; j < cols; j++) {
```


Edit with WPS Office

```
            printf("Element [%d][%d]: ", i + 1, j + 1);
            scanf("%d", &a[i][j]);
        }
    }

    // Input elements of the second matrix
    printf("Enter elements of the second matrix:\n");
    for(i = 0; i < rows; i++) {
        for(j = 0; j < cols; j++) {
            printf("Element [%d][%d]: ", i + 1, j + 1);
            scanf("%d", &b[i][j]);
        }
    }

    // Calculate the sum of the two matrices
    for(i = 0; i < rows; i++) {
        for(j = 0; j < cols; j++) {
            sum[i][j] = a[i][j] + b[i][j];  // Sum corresponding elements
        }
    }

    // Display the resulting matrix
    printf("Sum of the two matrices:\n");
    for(i = 0; i < rows; i++) {
        for(j = 0; j < cols; j++) {
            printf("%d ", sum[i][j]);
        }
        printf("\n");  // Move to the next line after each row
    }
}
```

```
Enter the number of rows and columns: 2 3
Enter elements of the first matrix:
Element [1][1]: 1
Element [1][2]: 2
Element [1][3]: 3
Element [2][1]: 4
Element [2][2]: 5
Element [2][3]: 6
Enter elements of the second matrix:
Element [1][1]: 7
Element [1][2]: 8
Element [1][3]: 9
Element [2][1]: 1
Element [2][2]: 2
Element [2][3]: 3
```


Edit with WPS Office

Sum of the two matrices:
8 10 12
5 7 9

23. Write a program for Subtraction of two matrices.

```
#include <stdio.h>  // Required for input-output functions

#define MAX 10  // Maximum size for the matrices

void main() {
    int a[MAX][MAX], b[MAX][MAX], difference[MAX][MAX];
    int rows, cols, i, j;

    // Input the number of rows and columns
    printf("Enter the number of rows and columns: ");
    scanf("%d %d", &rows, &cols);

    // Input elements of the first matrix
    printf("Enter elements of the first matrix:\n");
    for(i = 0; i < rows; i++) {
        for(j = 0; j < cols; j++) {
            printf("Element [%d][%d]: ", i + 1, j + 1);
            scanf("%d", &a[i][j]);
        }
    }

    // Input elements of the second matrix
    printf("Enter elements of the second matrix:\n");
    for(i = 0; i < rows; i++) {
        for(j = 0; j < cols; j++) {
            printf("Element [%d][%d]: ", i + 1, j + 1);
            scanf("%d", &b[i][j]);
        }
    }

    // Calculate the difference of the two matrices
    for(i = 0; i < rows; i++) {
        for(j = 0; j < cols; j++) {
            difference[i][j] = a[i][j] - b[i][j];  // Subtract corresponding elements
        }
    }

    // Display the resulting matrix
    printf("Difference of the two matrices:\n");
```

Edit with WPS Office

```
    for(i = 0; i < rows; i++) {
        for(j = 0; j < cols; j++) {
            printf("%d ", difference[i][j]);
        }
        printf("\n");  // Move to the next line after each row
    }
}
```

```
Enter the number of rows and columns: 2 3
Enter elements of the first matrix:
Element [1][1]: 10
Element [1][2]: 20
Element [1][3]: 30
Element [2][1]: 40
Element [2][2]: 50
Element [2][3]: 60
Enter elements of the second matrix:
Element [1][1]: 7
Element [1][2]: 8
Element [1][3]: 9
Element [2][1]: 1
Element [2][2]: 2
Element [2][3]: 3

Difference of the two matrices:
3 12 21
39 48 57
```

24. Write a program for Multiplication of two matrices.

```
#include <stdio.h>  // Required for input-output functions

#define MAX 10  // Maximum size for the matrices

void main() {
    int a[MAX][MAX], b[MAX][MAX], product[MAX][MAX];
    int rowsA, colsA, rowsB, colsB, i, j, k;

    // Input the dimensions for the first matrix
    printf("Enter the number of rows and columns for the first matrix: ");
    scanf("%d %d", &rowsA, &colsA);

    // Input elements of the first matrix
    printf("Enter elements of the first matrix:\n");
    for(i = 0; i < rowsA; i++) {
        for(j = 0; j < colsA; j++) {
```


Edit with WPS Office

```
            printf("Element [%d][%d]: ", i + 1, j + 1);
            scanf("%d", &a[i][j]);
        }
    }

    // Input the dimensions for the second matrix
    printf("Enter the number of rows and columns for the second matrix: ");
    scanf("%d %d", &rowsB, &colsB);

    // Check if multiplication is possible
    if(colsA != rowsB) {
        printf("Matrix multiplication not possible. Columns of first matrix must match rows of
second matrix.\n");
        return;  // Exit the program
    }

    // Input elements of the second matrix
    printf("Enter elements of the second matrix:\n");
    for(i = 0; i < rowsB; i++) {
        for(j = 0; j < colsB; j++) {
            printf("Element [%d][%d]: ", i + 1, j + 1);
            scanf("%d", &b[i][j]);
        }
    }

    // Initialize the product matrix to zero
    for(i = 0; i < rowsA; i++) {
        for(j = 0; j < colsB; j++) {
            product[i][j] = 0;  // Initialize each element to zero
        }
    }

    // Calculate the product of the two matrices
    for(i = 0; i < rowsA; i++) {
        for(j = 0; j < colsB; j++) {
            for(k = 0; k < colsA; k++) {
                product[i][j] += a[i][k] * b[k][j];  // Multiply and sum
            }
        }
    }

    // Display the resulting product matrix
    printf("Product of the two matrices:\n");
    for(i = 0; i < rowsA; i++) {
        for(j = 0; j < colsB; j++) {
            printf("%d ", product[i][j]);
```

Edit with WPS Office

```
        }
        printf("\n");  // Move to the next line after each row
    }
}
```

```
Enter the number of rows and columns for the first matrix: 2 3
Enter elements of the first matrix:
Element [1][1]: 1
Element [1][2]: 2
Element [1][3]: 3
Element [2][1]: 4
Element [2][2]: 5
Element [2][3]: 6
Enter the number of rows and columns for the second matrix: 3 2
Enter elements of the second matrix:
Element [1][1]: 7
Element [1][2]: 8
Element [2][1]: 9
Element [2][2]: 10
Element [3][1]: 11
Element [3][2]: 12

Product of the two matrices:
58 64
139 154
```


Edit with WPS Office

Lab Assignment-7 (Functions and Recursion)
Learn about types of functions-user defined functions , parameter passing call by value and recursion process

C Programming Functions and recursion

A function is a block of code that performs a specific task. Dividing complex problem into small components makes program easy to understand and use.

Types of functions in C programming

Depending on whether a function is defined by the user or already included in C compilers, there are two types of functions in C programming

There are two types of functions in C programming:

- Standard library functions
- User defined functions

Standard library functions

The standard library functions are built-in functions in C programming to handle tasks such as mathematical computations, I/O processing, string handling etc.
These functions are defined in the header file. When you include the header file, these functions are available for use. For example:
The printf() is a standard library function to send formatted output to the screen (display output on the screen). This function is defined in "stdio.h" header file.


Edit with WPS Office

There are other numerous library functions defined under "stdio.h", such as scanf(), fprintf(), getchar() etc. Once you include "stdio.h" in your program, all these functions are available for use.

User-defined functions

As mentioned earlier, C allow programmers to define functions. Such functions created by the user are called user-defined functions.
Depending upon the complexity and requirement of the program, you can create as many user-defined functions as you want.

How user-defined function works?

```
#include <stdio.h>
void functionName()
{
    ... .. ...
    ... .. ...
}

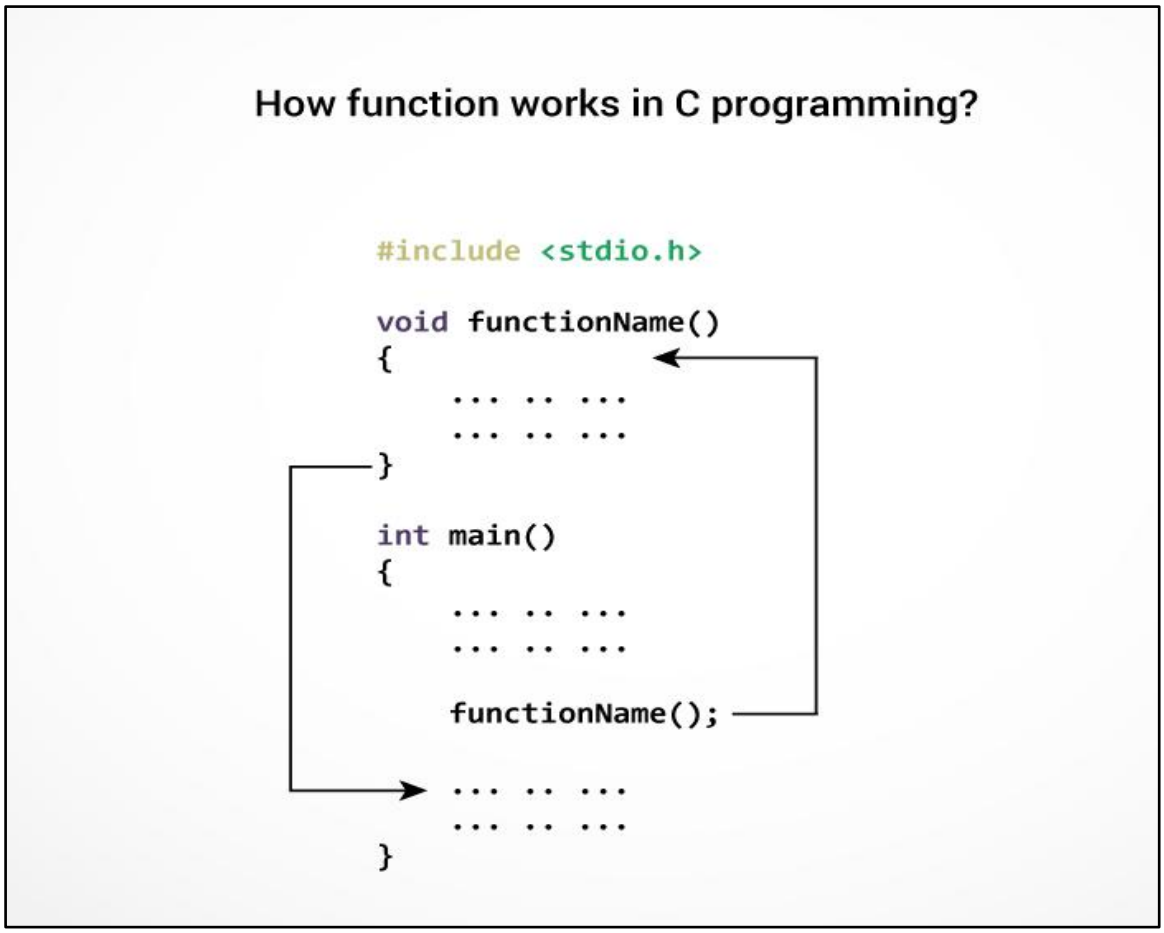
void main()
{
    ... .. ...
    ... .. ...

    functionName();

    ... .. ...
    ... .. ...
}
```

The execution of a C program begins from the main() function.
When the compiler encounters functionName(); inside the main function, control of the program jumps to
void functionName()
And, the compiler starts executing the codes inside the user-defined function.
The control of the program jumps to statement next to functionName(); once all the codes inside the function definition are executed

Edit with WPS Office



Example: User-defined function

Here is a example to add two integers. To perform this task, a user-defined function addNumbers() is defined.

```
#include <stdio.h>

int addNumbers(int a, int b);         // function prototype

int main()
{
   int n1,n2,sum;

   printf("Enters two numbers: ");
   scanf("%d %d",&n1,&n2);

   sum = addNumbers(n1, n2);         // function call

   printf("sum = %d",sum);

   return 0;
```

Edit with WPS Office

```
}


int addNumbers(int a,int b)         // function definition
{
   int result;
   result = a+b;
   return result;                  // return statement
}
```

Function prototype

A function prototype is simply the declaration of a function that specifies function's name, parameters and return type. It doesn't contain function body.
A function prototype gives information to the compiler that the function may later be used in the program.

Syntax of function prototype

returnType **functionName**(*type1 argument1, type2 argument2*,...);

In the above example, int addNumbers(int a, int b); is the function prototype which provides following information to the compiler:

1. name of the function is addNumbers()
2. return type of the function is int
3. two arguments of type int are passed to the function

The function prototype is not needed if the user-defined function is defined before the main() function.

Calling a function

Control of the program is transferred to the user-defined function by calling it

Syntax of function call

functionName(argument1, argument2, ...);

In the above example, function call is made using addNumbers(n1,n2); statement inside the main().


Edit with WPS Office

Function definition

Function definition contains the block of code to perform a specific task i.e. in this case, adding two numbers and returning it.

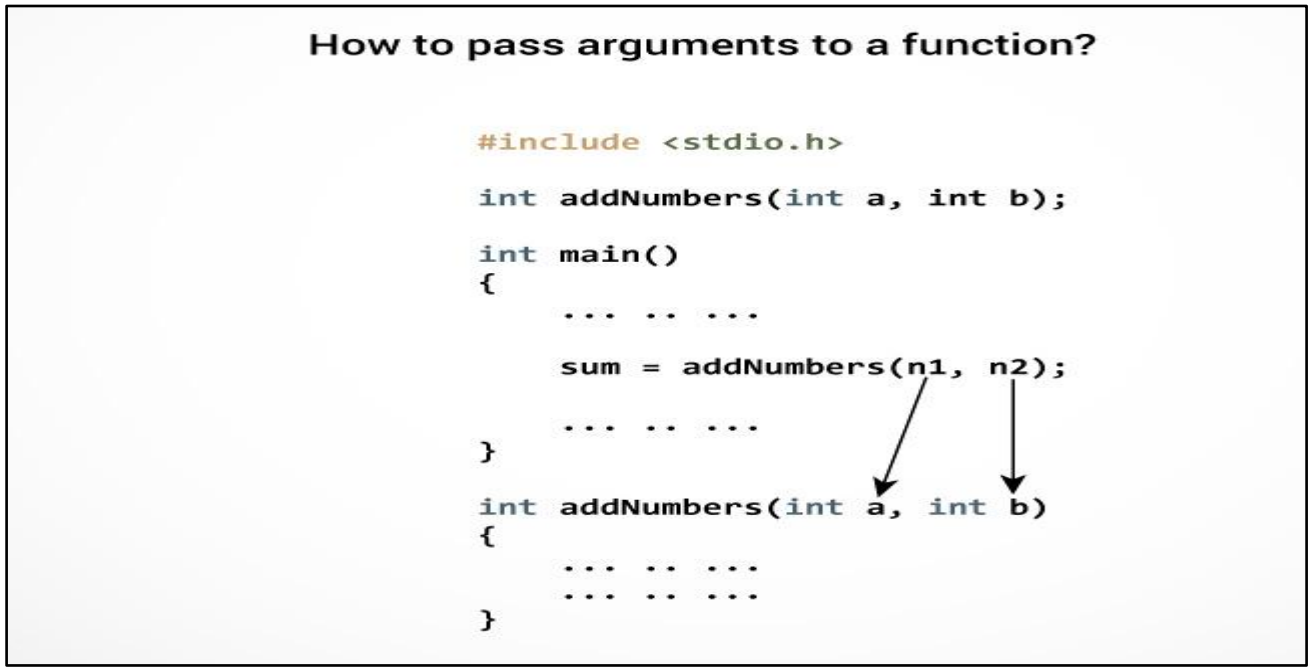
Syntax of function definition

```
returnType functionName(type1 argument1, type2 argument2, ...)
{
   //body of the function
}
```

When a function is called, the control of the program is transferred to the function definition. And, the compiler starts executing the codes inside the body of a function

Passing arguments to a function

In programming, argument refers to the variable passed to the function. In the above example, two variables *n1* and *n2* are passed during function call.
The parameters *a* and *b* accepts the passed arguments in the function definition. These arguments are called formal parameters of the function



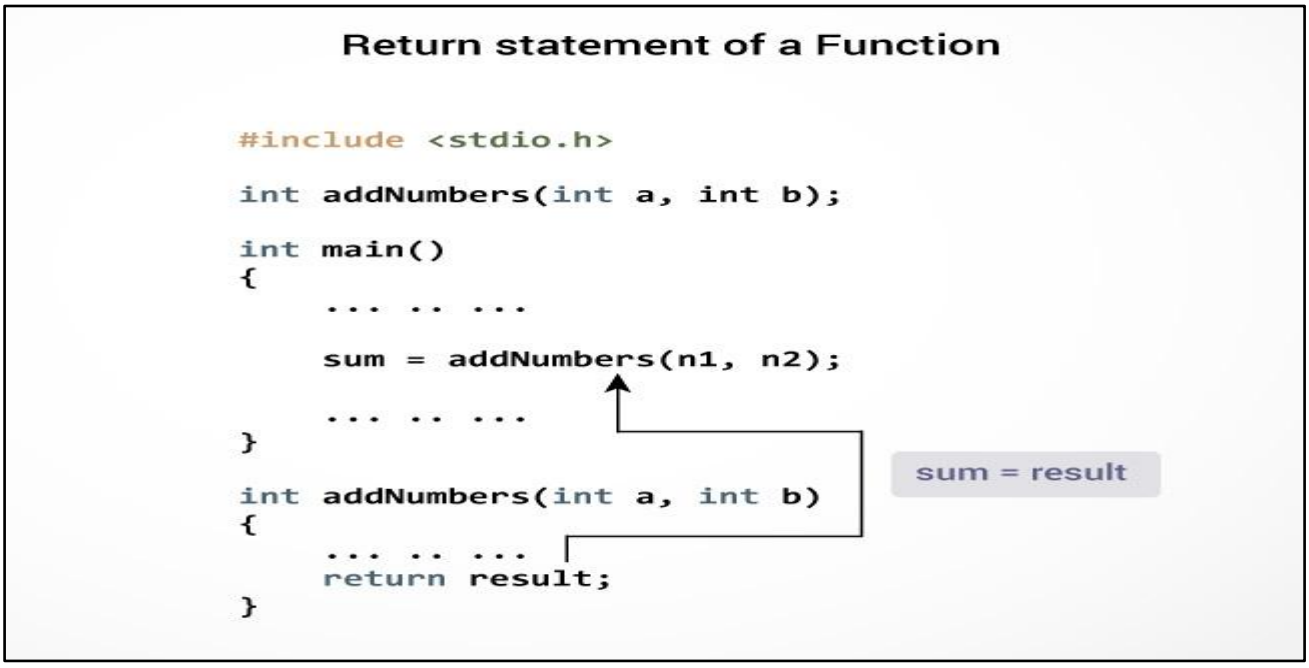
The type of arguments passed to a function and the formal parameters must match, otherwise the compiler throws error.

Edit with WPS Office

If *n1* is of char type, *a* also should be of char type. If *n2* is of float type, variable *b* also should be of float type.
A function can also be called without passing an argument

Return Statement

The return statement terminates the execution of a function and returns a value to the calling function. The program control is transferred to the calling function after return statement.
In the above example, the value of variable *result* is returned to the variable *sum* in the main() function



Syntax of return statement

return (expression);

For example,
return a;
return (a+b);

The type of value returned from the function and the return type specified in function prototype and function definition must match.

Types of User-defined Functions in C Programming

For better understanding of arguments and return value from the function, **user-defined functions** can be categorized as:

- Function with no arguments and no return value
- Function with no arguments and a return value

Edit with WPS Office

- Function with arguments and no return value
- Function with arguments and a return value.

Example #1: No arguments passed and no return Value

```
#include <stdio.h>
void checkPrimeNumber();

int main()
{
   checkPrimeNumber();    // no argument is passed to prime()
   return 0;
}

// return type of the function is void because no value is returned from the function
void checkPrimeNumber()
{
   int n, i, flag=0;

   printf("Enter a positive integer: ");
   scanf("%d",&n);

   for(i=2; i <= n/2; ++i)
   {
      if(n%i == 0)
      {
         flag = 1;
      }
   }
   if (flag == 1)
      printf("%d is not a prime number.", n);
   else
      printf("%d is a prime number.", n);
}
```

The checkPrimeNumber() function takes input from the user, checks whether it is a prime number or not and displays it on the screen.
The empty parentheses in checkPrimeNumber(); statement inside the main() function indicates that no argument is passed to the function.
The return type of the function is void. Hence, no value is returned from the function

Example #2: No arguments passed but a return value

```
#include <stdio.h>
int getInteger();
```


Edit with WPS Office

```
int main()
{
    int n, i, flag = 0;

    // no argument is passed to the function
    // the value returned from the function is assigned to n
    n = getInteger();

    for(i=2; i<=n/2; ++i)
    {
        if(n%i==0){
            flag = 1;
            break;
        }
    }
    if (flag == 1)
        printf("%d is not a prime number.", n);
    else
        printf("%d is a prime number.", n);

    return 0;
}

// getInteger() function returns integer entered by the user
int getInteger()
{
    int n;

    printf("Enter a positive integer: ");
    scanf("%d",&n);

    return n;
}
```

The empty parentheses in n = getInteger(); statement indicates that no argument is passed to the function. And, the value returned from the function is assigned to *n*.Here, the getInteger() function takes input from the user and returns it. The code to check whether a number is prime or not is inside the main() function

Example #3: Argument passed but no return value

```
#include <stdio.h>
void checkPrimeAndDisplay(int n);

int main()
{
```


Edit with WPS Office

```
    int n;

    printf("Enter a positive integer: ");
    scanf("%d",&n);

    // n is passed to the function
    checkPrimeAndDisplay(n);

    return 0;
}

// void indicates that no value is returned from the function
void checkPrimeAndDisplay(int n)
{
    int i, flag = 0;

    for(i=2; i <= n/2; ++i)
    {
        if(n%i == 0){
            flag = 1;
            break;
        }
    }
    if(flag == 1)
        printf("%d is not a prime number.",n);
    else
        printf("%d is a prime number.", n);
}
```

The integer value entered by the user is passed to checkPrimeAndDisplay() function.
Here, the checkPrimeAndDisplay() function checks whether the argument passed is a prime number or not and displays the appropriate message

Example #4: Argument passed and a return value

```
#include <stdio.h>
int checkPrimeNumber(int n);

int main()
{
```


Edit with WPS Office

```
    int n, flag;

    printf("Enter a positive integer: ");
    scanf("%d",&n);

    // n is passed to the checkPrimeNumber() function
    // the value returned from the function is assigned to flag variable
    flag = checkPrimeNumber(n);

    if(flag==1)
        printf("%d is not a prime number",n);
    else
        printf("%d is a prime number",n);

    return 0;
}

// integer is returned from the function
int checkPrimeNumber(int n)
{
    /* Integer value is returned from function checkPrimeNumber() */
    int i;

    for(i=2; i <= n/2; ++i)
    {
        if(n%i == 0)
            return 1;
    }

    return 0;
}
```

The input from the user is passed to checkPrimeNumber() function.
The checkPrimeNumber() function checks whether the passed argument is prime or not. If the passed argument is a prime number, the function returns 0. If the passed argument is a non-prime number, the function returns 1. The return value is assigned to *flag* variable.
Then, the appropriate message is displayed from the main() function.

Which approach is better?

Well, it depends on the problem you are trying to solve. In case of this problem, the last approach is better.


Edit with WPS Office

A function should perform a specific task. The checkPrimeNumber() function doesn't take input from the user nor it displays the appropriate message. It only checks whether a number is prime or not, which makes code modular, easy to understand and debug.

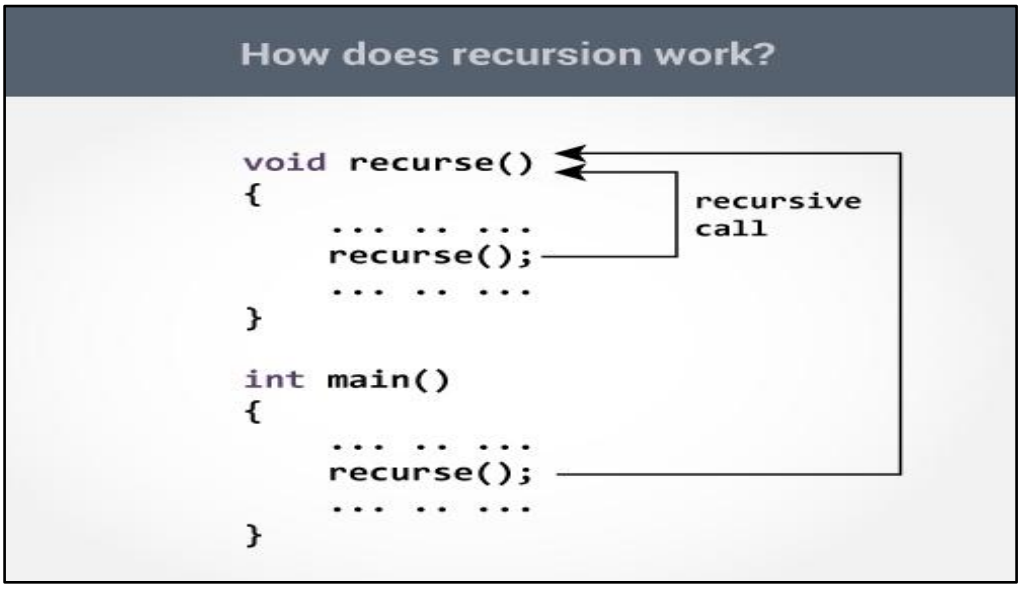
C Programming Recursion

A function that calls itself is known as a recursive function. And, this technique is known as recursion

How recursion works?

```
void recurse()
{
    ... .. ...
    recurse();
    ... .. ...
}

int main()
{
    ... .. ...
    recurse();
    ... .. ...
}
```



The recursion continues until some condition is met to prevent it.
To prevent infinite recursion, if...else statement (or similar approach) can be used where one branch makes the recursive call and other doesn't.

Edit with WPS Office

Example: Sum of Natural Numbers Using Recursion

```
#include <stdio.h>
int sum(int n);

int main()
{
    int number, result;

    printf("Enter a positive integer: ");
    scanf("%d", &number);

    result = sum(number);

    printf("sum = %d", result);
    return 0;
}

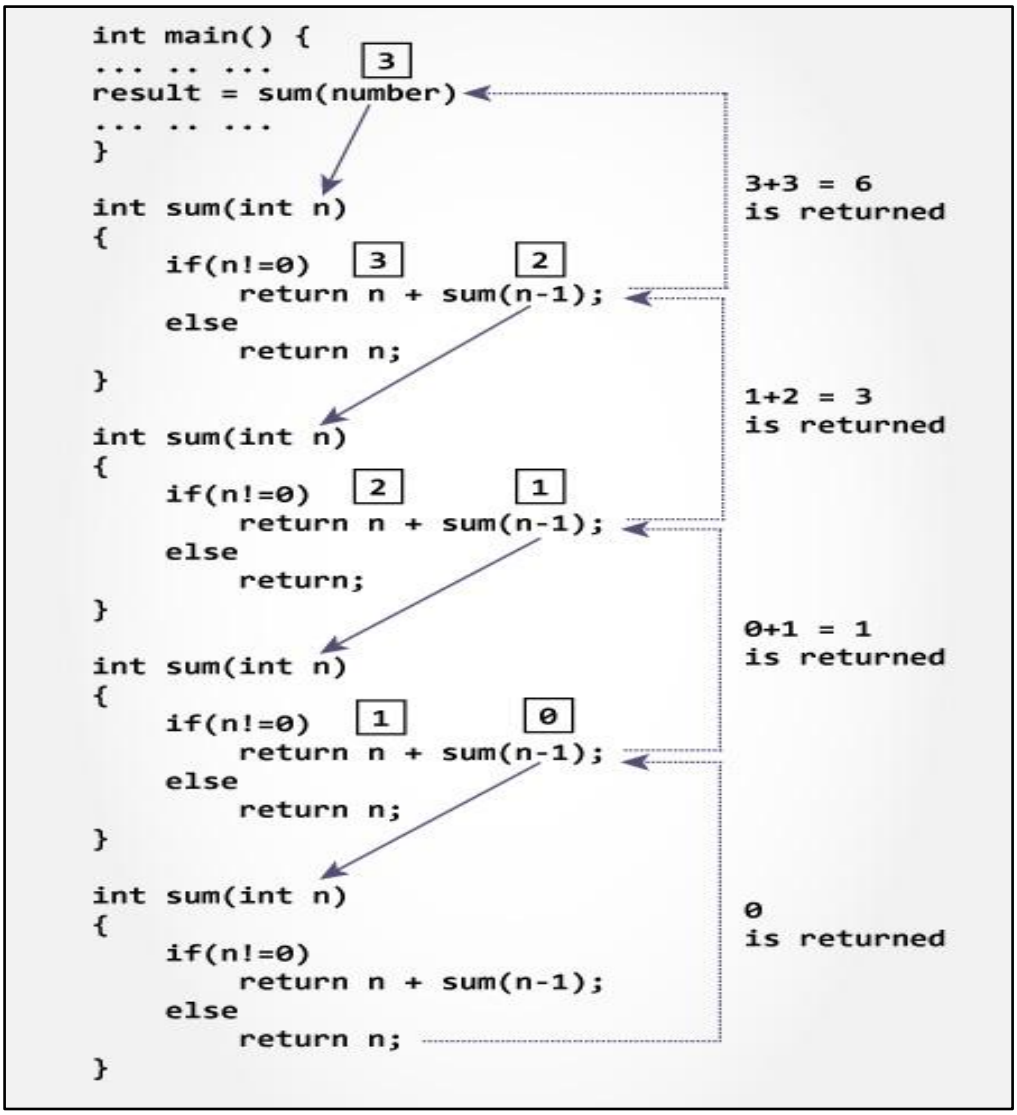
int sum(int num)
{
    if (num!=0)
        return num + sum(num-1); // sum() function calls itself
    else
        return num;
}
```

Output

```
Enter a positive integer:3
sum = 6
```

Initially, the sum() is called from the main() function with *number* passed as an argument.
Suppose, the value of *num* is 3 initially. During next function call, 2 is passed to the sum() function. This process continues until *num* is equal to 0.
When *num* is equal to 0, the if condition fails and the else part is executed returning the sum of integers to the main() function.


Edit with WPS Office



Advantages and Disadvantages of Recursion

Recursion makes program elegant and cleaner. All algorithms can be defined recursively which makes it easier to visualize and prove.

If the speed of the program is vital then, you should avoid using recursion. Recursions use more memory and are generally slow. Instead, you can use loop.

25. Write a program to find Prime numbers between a given ranges using function.

```
#include <stdio.h>  // Required for input-output functions

// Function to check if a number is prime
```

Edit with WPS Office

```
int isPrime(int num) {
    if (num <= 1) {
        return 0;  // 0 and 1 are not prime numbers
    }
    for (int i = 2; i * i <= num; i++) {  // Check divisibility up to the square root of num
        if (num % i == 0) {
            return 0;  // Not a prime number
        }
    }
    return 1;  // Prime number
}

void main() {
    int lower, upper;

    // Input the range
    printf("Enter the lower and upper limits of the range: ");
    scanf("%d %d", &lower, &upper);

    printf("Prime numbers between %d and %d are:\n", lower, upper);
    for (int i = lower; i <= upper; i++) {
        if (isPrime(i)) {
            printf("%d ", i);  // Print the prime number
        }
    }
    printf("\n");  // New line after printing all prime numbers
}
```

```
Enter the lower and upper limits of the range: 10 50
Prime numbers between 10 and 50 are:
11 13 17 19 23 29 31 37 41 43 47
```

26. Write a program to find sum of n Natural Numbers using function.

```
#include <stdio.h>  // Required for input-output functions

// Function to calculate the sum of first n natural numbers
int sumOfNaturalNumbers(int n) {
    return (n * (n + 1)) / 2;  // Formula: n(n + 1) / 2
}

void main() {
```

Edit with WPS Office

```
    int n;

    // Input the value of n
    printf("Enter a natural number: ");
    scanf("%d", &n);

    // Validate the input
    if (n < 1) {
        printf("Please enter a natural number greater than 0.\n");
    } else {
        // Calculate and display the sum
        int sum = sumOfNaturalNumbers(n);
        printf("The sum of the first %d natural numbers is: %d\n", n, sum);
    }
}
```

```
Enter a natural number: 5
The sum of the first 5 natural numbers is: 15
```

27. **Write a program to calculate Factorial of a number using recursion.**

```
#include <stdio.h>  // Required for input-output functions

// Function to calculate factorial using recursion
long long factorial(int n) {
    if (n <= 1) {
                        return 1;  // Base case: 0! and 1! are both 1
                    }
  return n * factorial(n - 1);  // Recursive case
}

void main() {
            int n;

             // Input the number
```

Edit with WPS Office

```
    printf("Enter a non-negative integer: ");
    scanf("%d", &n);

    // Validate the input
    if (n < 0) {
        printf("Factorial is not defined for negative numbers.\n");
    } else {
        // Calculate and display the factorial
        long long result = factorial(n);
        printf("The factorial of %d is: %lld\n", n, result);
    }
}
```

Output:

Enter a non-negative integer: 5
The factorial of 5 is: 120

Lab Assignment-8 (String and Character Array)
String declaration, initialization, different operations on string like copy, concatenate, reverse ,length, and comparison using string function and without using string function.

String and Character Array

String is a sequence of characters that is treated as a single data item and terminated by null character '\0'. Remember that C language does not support strings as a data type. A **string** is actually one-dimensional array of characters in C language. These are often used to create meaningful and readable programs.
For example: The string "hello world" contains 12 characters including '\0' character which is automatically added by the compiler at the end of the string

Declaring and Initializing a string variables

There are different ways to initialize a character array variable.

```
char name[12] = "SKIT JAIPUR";       // valid character array initialization

char name[10] = {'L','e','s','s','o','n','s','\0'};     // valid initialization
```


Edit with WPS Office

Remember that when you initialize a character array by listing all of its characters separately then you must supply the '\0' character explicitly.
Some examples of illegal initialization of character array are

```
char ch[3] = "hell";    // Illegal character array initialization

char str[4];
str = "hell";   // Illegal character array initialization
```

String Input and Output

Input function scanf() can be used with **%s** format specifier to read a string input from the terminal(keyboard). But there is one problem with scanf() function, it terminates its input on the first white space it encounters. Therefore if you try to read an input string "Hello World" using scanf() function, it will only read **Hello** and terminate after encountering white spaces.
However, C supports a format specification known as the **edit set conversion code %[..]** that can be used to read a line containing a variety of characters, including white spaces.

Here are the example use of **edit set conversion code %[..]**

```
#include<stdio.h>
#include<string.h>

void main()
{
   char str[20];
   printf("Enter a string");
   scanf("%[^\n]", &str);  //scanning the whole string, including the white spaces
   printf("%s", str);
}
```

Input: skit engg college jaipur rajasthan
Output: skit engg college jaipur rajasthan
Note*: Another method to read character string with white spaces from terminal is by using the gets() function.

```
char text[20];
gets(text);
printf("%s", text);
```

String Handling Functions

C language supports a large number of string handling functions that can be used to carry out many of the string manipulations. These functions are packaged in **string.h** library. Hence, you must include **string.h** header file in your programs to use these functions.


Edit with WPS Office

The following are the most commonly used string handling functions.

Function Name	Description
strcat()	It is used to concatenate(combine) two strings
strlen()	It is used to show length of a string
strrev()	It is used to show reverse of a string
strcpy()	Copies one string into another
strcmp()	It is used to compare two string
strlwr()	It converts a string to lowercase
strupr()	It converts a string to uppercase
strncat()	It appends first n characters of a string at the end of another string
strncpy()	It copies first n characters of one string into another string
strncmp()	It compares first n characters of two strings
strcmpi()	It compares two strings without regard to case ("i" denotes that this function ignores case)
stricmp()	It compares two strings without regard to case (identical to strcmpi)
strdup()	Used for duplicating a string
strchr()	Find out first occurrence of given character in a string
strrchr()	Find out last occurrences of a given character in string
strstr()	Finds first occurrences of a given string in another string
strset()	It sets all characters of string to a given character
strnset()	It sets first n characters of string to a given character

Edit with WPS Office

28. Write a program to convert a string from lowercase to uppercase.

```
#include <stdio.h>  // Required for input-output functions

void convertToUppercase(char str[]) {
    int i = 0;
    while (str[i] != '\0') {  // Loop until the end of the string
        if (str[i] >= 'a' && str[i] <= 'z') {  // Check if the character is lowercase
            str[i] = str[i] - 'a' + 'A';  // Convert to uppercase
        }
        i++;
    }
}

void main() {
    char str[100];  // Array to store the input string

    // Input the string
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);  // Using fgets to read the string including spaces

    // Convert to uppercase
    convertToUppercase(str);

    // Display the converted string
    printf("String in uppercase: %s\n", str);
}
Enter a string: Hello World!
String in uppercase: HELLO WORLD!
```


Edit with WPS Office

29. **Write a program to find total number of alphabets, digits or special characters in a string.**

```
#include <stdio.h>  // Required for input-output functions

void countCharacters(char str[], int *alphaCount, int *digitCount, int *specialCount) {
    int i = 0;
    while (str[i] != '\0') {  // Loop until the end of the string
        if ((str[i] >= 'a' && str[i] <= 'z') || (str[i] >= 'A' && str[i] <= 'Z')) {
            (*alphaCount)++;  // Increment alphabet count
        } else if (str[i] >= '0' && str[i] <= '9') {
            (*digitCount)++;  // Increment digit count
        } else if (str[i] != ' ' && str[i] != '\n') {  // Count special characters (ignoring spaces and newline)
            (*specialCount)++;  // Increment special character count
        }
        i++;
    }
}

void main() {
    char str[100];  // Array to store the input string
    int alphabets = 0, digits = 0, specials = 0;

    // Input the string
    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);  // Using fgets to read the string including spaces

    // Count characters
    countCharacters(str, &alphabets, &digits, &specials);

    // Display the counts
    printf("Total alphabets: %d\n", alphabets);
    printf("Total digits: %d\n", digits);
    printf("Total special characters: %d\n", specials);
}
```

```
Enter a string: Hello World! 123
Total alphabets: 10
Total digits: 3
```

Edit with WPS Office

Total special characters: 2

Lab Assignment-9 (Pointers)
Learn about pointers , arithmetic operations on pointers, pointer with array, pointer with string, pointer to pointer

C Programming Pointers

Pointers are powerful features of C and (C++) programming that differentiates it from other popular programming languages like: Java and Python.
Pointers are used in C program to access the memory and manipulate the address.

Address in C

Before you get into the concept of pointers, let's first get familiar with address in C.
If you have a variable *var* in your program, &var will give you its address in the memory, where & is commonly called the reference operator.
You must have seen this notation while using scanf() function. It was used in the function to store the user inputted value in the address of *var*.
scanf("%d", &var);

```
/* Example to demonstrate use of reference operator in C programming. */
#include <stdio.h>
int main()
{
  int var = 5;
  printf("Value: %d\n", var);
  printf("Address: %u", &var);  //Notice, the ampersand(&) before var.
  return 0;
}
```

Output
Value: 5
Address: 2686778

Note: In above source code, value 5 is stored in the memory location 2686778. *var* is just the name given to that location

Edit with WPS Office

Pointer variables

In C, there is a special variable that stores just the address of another variable. It is called Pointer variable or, simply, a pointer.
Declaration of Pointer
data_type* pointer_variable_name;
int* p;

Above statement defines, *p* as pointer variable of type int.

Reference operator (&) and Dereference operator (*)

As discussed, & is called reference operator. It gives you the address of a variable.
Likewise, there is another operator that gets you the value from the address, it is called a dereference operator (*)..
Note: The * sign when declaring a pointer is not a dereference operator. It is just a similar notation that creates a pointer.

Example To Demonstrate Working of Pointers

Source code to demonstrate, handling of pointers in C program

```
#include <stdio.h>
void main()
{
   int* pc;
   int c;
   c = 22;
   printf("Address of c: %u\n", &c);
   printf("Value of c: %d\n\n", c);
   pc = &c;
   printf("Address of pointer pc: %u\n", pc);
   printf("Content of pointer pc: %d\n\n", *pc);
   c = 11;
   printf("Address of pointer pc: %u\n", pc);
   printf("Content of pointer pc: %d\n\n", *pc);
  *pc = 2;
   printf("Address of c: %u\n", &c);
   printf("Value of c: %d\n\n", c);
   getch();
}
```

Output

```
Address of c: 2686784
Value of c: 22
Address of pointer pc: 2686784
Content of pointer pc: 22
```

Edit with WPS Office

Address of pointer pc: 2686784
Content of pointer pc: 11
Address of c: 2686784
Value of c: 2

Explanation of program

1 int* pc; creates a pointer *pc* and int c; creates a normal variable *c*.
Since *pc* and *c* are both not initialized, pointer *pc* points to either no address or a random address. Likewise, variable *c* is assigned an address but contains a random/garbage value.
2 c=22; assigns 22 to the variable c, i.e.,22 is stored in the memory location of variable *c*.
Note that, when printing &c (address of c), we use %u rather than %d since address is usually expressed as an unsigned integer (always positive).
3 pc=&c; assigns the address of variable to *c* to the pointer *pc*.
When printing, you see value of *pc* is the same as the address of *c* and the content of *pc* (*pc) is 22 as well.
4 c=11; assigns 11 to variable *c*.
We assign a new value to *c* to see its effect on pointer *pc*.
5 Since, pointer *pc* points to the same address as *c*, value pointed by pointer *pc* is 11 as well.
Printing the address and content of *pc* shows the updated content as 11.
6 *pc=2; changes the contents of the memory location pointed by pointer *pc* to 2.
Since the address of pointer *pc* is same as address of *c*, value of *c* also changes to 2

Common mistakes when working with pointers

Suppose, you want pointer pc to point to the address of c. Then,

```
int c, *pc;
pc = c;          // Wrong! pc is address whereas, c is not an address.
*pc = &c;        // Wrong! *pc is the value pointed by address whereas, &c is an address..
pc = &c;         // Correct! pc is an address and, &c is also an address
*pc = c;         // Correct! *pc is the value pointed by address and, c is also a value
```

In both cases, pointer pc is not pointing to the address of c.

C Programming Pointers and Arrays

Arrays are closely related to pointers in C programming but the important difference between them is that, a pointer variable takes different addresses as value whereas, in case of array it is fixed.
This can be demonstrated by an example

```
#include <stdio.h>
int main()
{
  char charArr[4];
  int i;
```


Edit with WPS Office

```
  for(i = 0; i < 4; ++i)
  {
    printf("Address of charArr[%d] = %u\n", i, &charArr[i]);
  }

  return 0;
}
```

When you run the program, the output will be:
Address of charArr[0] = 28ff44
Address of charArr[1] = 28ff45
Address of charArr[2] = 28ff46
Address of charArr[3] = 28ff47

Note: You may get different address of an array.
Notice, that there is an equal difference (difference of 1 byte) between any two consecutive elements of array *charArr*.
But, since pointers just point at the location of another variable, it can store any address

Relation between Arrays and Pointers

Consider an array:
int arr[4];

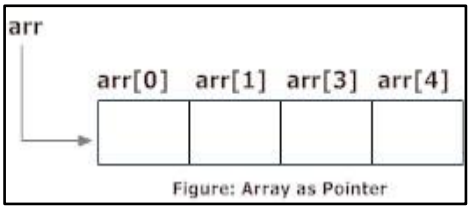


Figure: Array as Pointer

In C programming, name of the array always points to address of the first element of an array.
In the above example, *arr* and *&arr[0]* points to the address of the first element.
&arr[0] is equivalent to arr
Since, the addresses of both are the same, the values of arr and &arr[0] are also the same.
arr[0] is equivalent to *arr (value of an address of the pointer)

&arr[1] is equivalent to (arr + 1) AND, arr[1] is equivalent to *(arr + 1).

Edit with WPS Office

&arr[2] is equivalent to (arr + 2) AND, arr[2] is equivalent to *(arr + 2).
&arr[3] is equivalent to (arr + 3) AND, arr[3] is equivalent to *(arr + 3).
.
.
&arr[i] is equivalent to (arr + i) AND, arr[i] is equivalent to *(arr + i).

In C, you can declare an array and can use pointer to alter the data of an array.

Example: Program to find the sum of six numbers with arrays and pointers

```
#include <stdio.h>
#include<conio.h>
int main()
{
  int i, classes[6],sum = 0;
  printf("Enter 6 numbers:\n");
  for(i = 0; i < 6; ++i)
  {
      // (classes + i) is equivalent to &classes[i]
      scanf("%d",(classes + i));

      // *(classes + i) is equivalent to classes[i]
      sum += *(classes + i);
  }
  printf("Sum = %d", sum);
  return 0;
}
```

Output

```
Enter 6 numbers:
2
3
4
5
3
4
Sum = 21
```


Edit with WPS Office

C Call by Reference: Using pointers

When a pointer is passed as an argument to a function, address of the memory location is passed instead of the value.
This is because, pointer stores the location of the memory, and not the value

Example of Pointer And Functions

Program to swap two number using call by reference.

```
#include <stdio.h>
void s
wap(int *n1, int *n2);

int main()
{
   int num1 = 5, num2 = 10;

   // address of num1 and num2 is passed to the swap function
   swap( &num1, &num2);
   printf("Number1 = %d\n", num1);
   printf("Number2 = %d", num2);
   return 0;
}

void swap(int * n1, int * n2)
{
   // pointer n1 and n2 points to the address of num1 and num2 respectively
   int temp;
   temp = *n1;
   *n1 = *n2;
   *n2 = temp;
}
```

Output

```
Number1 = 10
Number2 = 5
```

Explanation of program

The address of *num1* and *num2* are passed to the function swap() and the pointers **n1* and **n2* accept those values.
Now, pointer *n1* contains the address in memory of variable *num1*. And, *n2* contains the address of variable *num2*.
When the value of pointers are changed, the value in the pointed memory location is also changed.
Hence, changes made to *n1 and **n2* are reflected in *num1* and *num2* in the main() function.
This technique is known as call by reference in C programming.

Edit with WPS Office

30. Write a program to create, initialize and use of pointers.

```
#include <stdio.h>  // Required for input-output functions

void main() {
   int num = 10;            // An integer variable
   int *ptr;               // Declaration of a pointer to an integer

   // Initialize the pointer to the address of the variable num
   ptr = &num;             // ptr now points to num

   // Display the value of num using the pointer
   printf("Value of num: %d\n", num);
   printf("Value of num using pointer: %d\n", *ptr);  // Dereference pointer to get the value

   // Modify the value of num using the pointer
   *ptr = 20;              // Changing the value at the address ptr points to

   // Display the updated value of num
   printf("Updated value of num: %d\n", num);
   printf("Value of num using pointer after update: %d\n", *ptr);  // Dereference pointer
again
}
```


Edit with WPS Office

31. Write a program to perform all arithmetic operations using pointers.

```
#include <stdio.h>  // Required for input-output functions

void performArithmeticOperations(int *a, int *b, int *sum, int *diff, int *prod, float *quotient, int
*modulus) {
    *sum = *a + *b;                // Addition
    *diff = *a - *b;               // Subtraction
    *prod = *a * *b;               // Multiplication
    if (*b != 0) {                 // Check for division by zero
        *quotient = (float)(*a) / (*b);  // Division
        *modulus = *a % *b;        // Modulus
    } else {
        *quotient = 0;             // Handle division by zero case
        *modulus = 0;              // Handle modulus by zero case
    }
}

void main() {
    int num1, num2;                // Variables to store input numbers
    int sum, diff, prod, modulus; // Variables to store results
    float quotient;                // Variable to store quotient

    // Input two numbers
    printf("Enter first number: ");
    scanf("%d", &num1);
    printf("Enter second number: ");
    scanf("%d", &num2);

    // Perform arithmetic operations using pointers
    performArithmeticOperations(&num1, &num2, &sum, &diff, &prod, &quotient, &modulus);
```

Edit with WPS Office

```
    // Display results
    printf("Addition: %d\n", sum);
    printf("Subtraction: %d\n", diff);
    printf("Multiplication: %d\n", prod);
    if (num2 != 0) {
        printf("Division: %.2f\n", quotient);
        printf("Modulus: %d\n", modulus);
    } else {
        printf("Division: Undefined (division by zero)\n");
        printf("Modulus: Undefined (division by zero)\n");
    }
}
```

```
Enter first number: 15
Enter second number: 0
Addition: 15
Subtraction: 15
Multiplication: 0
Division: Undefined (division by zero)
Modulus: Undefined (division by zero)
```


Edit with WPS Office

32. Write a program to swap two numbers using call by references.

```
#include <stdio.h>  // Required for input-output functions

// Function to swap two numbers using pointers (call by reference)
void swap(int *a, int *b) {
    int temp;       // Temporary variable for swapping
    temp = *a;      // Store the value at address a in temp
    *a = *b;        // Assign the value at address b to address a
    *b = temp;      // Assign the value stored in temp to address b
}

void main() {
    int num1, num2;  // Variables to store the numbers

    // Input two numbers
    printf("Enter first number: ");
    scanf("%d", &num1);
    printf("Enter second number: ");
    scanf("%d", &num2);

    // Display numbers before swapping
    printf("Before swapping: num1 = %d, num2 = %d\n", num1, num2);

    // Call the swap function
    swap(&num1, &num2);

    // Display numbers after swapping
    printf("After swapping: num1 = %d, num2 = %d\n", num1, num2);
}

Enter first number: 10
Enter second number: 20
Before swapping: num1 = 10, num2 = 20
After swapping: num1 = 20, num2 = 10
```


Edit with WPS Office

33. Write a program to use the concepts of double pointers (Pointers to Pointers).

```
#include <stdio.h>  // Required for input-output functions
#include <stdlib.h> // Required for dynamic memory allocation

// Function to allocate memory for an integer and return its address using double pointer
void allocateMemory(int **ptr) {
    *ptr = (int *)malloc(sizeof(int)); // Allocate memory for an integer
    if (*ptr == NULL) {
        printf("Memory allocation failed!\n");
        exit(1); // Exit if memory allocation fails
    }
}

// Function to modify the value pointed by the double pointer
void modifyValue(int **ptr) {
    **ptr = 42; // Set the value of the integer to 42
}

void main() {
    int *singlePtr; // Declare a single pointer
    int **doublePtr; // Declare a double pointer
    doublePtr = &singlePtr; // Make doublePtr point to singlePtr

    // Allocate memory for the integer using the double pointer
    allocateMemory(doublePtr);

    // Modify the value using the double pointer
    modifyValue(doublePtr);

    // Display the value
    printf("Value pointed by singlePtr: %d\n", *singlePtr);

    // Free allocated memory
    free(singlePtr);
}

Value pointed by singlePtr: 42
```

Lab Assignment-10 (Structure and Union)


Edit with WPS Office

Learn about structure ,declaration of structure, initialization, nested structure, structure pointer, array of structure , coping structure , passing structure to the user defined functions and differentiate structure and union

C Programming Structure

Structure is a collection of variables of different types under a single name
For example: You want to store some information about a person: his/her name, citizenship number and salary. You can easily create different variables *name, citNo, salary* to store these information separately.
However, in the future, you would want to store information about multiple persons. Now, you'd need to create different variables for each information per person: *name1, citNo1, salary1, name2, citNo2, salary2*
You can easily visualize how big and messy the code would look. Also, since no relation between the variables (information) would exist, it's going to be a daunting task.
A better approach will be to have a collection of all related information under a single name Person, and use it for every person. Now, the code looks much cleaner, readable and efficient as well.
This collection of all related information under a single name Person is a structure.

Structure Definition in C

Keyword struct is used for creating a structure

Syntax of structure

```
struct structure_name
{
   data_type member1;
   data_type member2;
   .
   .
   data_type memeber;
};
```

Note: Don't forget the semicolon }; in the ending line

We can create the structure for a person as mentioned above as:

```
struct person
{
   char name[50];
   int citNo;
   float salary;
};
```

Edit with WPS Office

This declaration above creates the derived data type **struct person**

Structure variable declaration

When a structure is defined, it creates a user-defined type but, no storage or memory is allocated.
For the above structure of a person, variable can be declared as:

```
struct person
{
    char name[50];
    int citNo;
    float salary;
};

int main()
{
    struct person person1, person2, person3[20];
    return 0;
}
```

Another way of creating a structure variable is:

```
struct person
{
    char name[50];
    int citNo;
    float salary;
} person1, person2, person3[20];
```

In both cases, two variables *person1*, *person2* and an array *person3* having 20 elements of type **struct person** are created.

Accessing members of a structure

There are two types of operators used for accessing members of a structure.

1. Member operator(.)
2. Structure pointer operator(->) (is discussed in structure and pointers tutorial)

Any member of a structure can be accessed as:

structure_variable_name.member_name

Suppose, we want to access salary for variable *person2*. Then, it can be accessed as:
person2.salary


Edit with WPS Office

Example of structure

Write a C program to add two distances entered by user. Measurement of distance should be in inch and feet. (Note: 12 inches = 1 foot)

```
#include <stdio.h>
struct Distance
{
    int feet;
    float inch;
} dist1, dist2, sum;

int main()
{
    printf("1st distance\n");

    // Input of feet for structure variable dist1
    printf("Enter feet: ");
    scanf("%d", &dist1.feet);

    // Input of inch for structure variable dist1
    printf("Enter inch: ");
    scanf("%f", &dist1.inch);

    printf("2nd distance\n");

    // Input of feet for structure variable dist2
    printf("Enter feet: ");
    scanf("%d", &dist2.feet);

    // Input of feet for structure variable dist2
```

Edit with WPS Office

```
    printf("Enter inch: ");
    scanf("%f", &dist2.inch);

    sum.feet = dist1.feet + dist2.feet;
    sum.inch = dist1.inch + dist2.inch;

    if (sum.inch > 12)
    {
            //If inch is greater than 12, changing it to feet.
        ++sum.feet;
        sum.inch = sum.inch - 12;
    }

    // printing sum of distance dist1 and dist2
    printf("Sum of distances = %d\'-%.1f\"", sum.feet, sum.inch);
    return 0;
}
```

Output

```
1st distance
Enter feet: 12
Enter inch: 7.9
2nd distance
Enter feet: 2
Enter inch: 9.8
Sum of distances = 15'-5.7"
```

Keyword typedef while using structure

Writing struct structure_name variable_name; to declare a structure variable isn't intuitive as to what it signifies, and takes some considerable amount of development time.
So, developers generally use typedef to name the structure as a whole. For Example

```
typedef struct complex
{
  int imag;
  float real;
} comp;

int main()
{
  comp comp1, comp2;
}
```

Here, typedef keyword is used in creating a type *comp* (which is of type as **struct complex**).
Then, two structure variables *comp1* and *comp2* are created by this *comp* type.


Edit with WPS Office

Structures within structures

Structures can be nested within other structures in C programming

```
struct complex
{
 int imag_value;
 float real_value;
};

struct number
{
   struct complex comp;
   int real;
} num1, num2;
```

Suppose, you want to access *imag_value* for *num2* structure variable then, following structure member is used.

num2.comp.imag_value

Union in C

Unions are quite similar to structures in C. Like structures, unions are also derived types.

```
union car
{
  char name[50];
  int price;
};
```

Defining a union is as easy as replacing the keyword **struct** with the keyword **union**.

How to create union variables?

Union variables can be created in similar manner as structure variables

```
union car
{
  char name[50];
  int price;
} car1, car2;
```

OR

```
union car
```

Edit with WPS Office

```
{
  char name[50];
  int price;
};

int main()
{
  union car car1, car2;
  return 0;
}
```

In both cases, union variables *car1*, *car2* of type **union car** is created

Accessing members of a union

Again, the member of unions can be accessed in similar manner as structures.

```
union test
{
    int a;
    float b;
    char c;
}t;

t.a;    //to access members of union t
t.b;
t.c;
```

Example:

```
#include <stdio.h>
union item
{
    int a;
    float b;
    char ch;
};
int main( )
{
    union item it;
    it.a = 12;
    it.b = 20.2;
    it.ch = 'z';
    printf("%d\n", it.a);
```

Edit with WPS Office

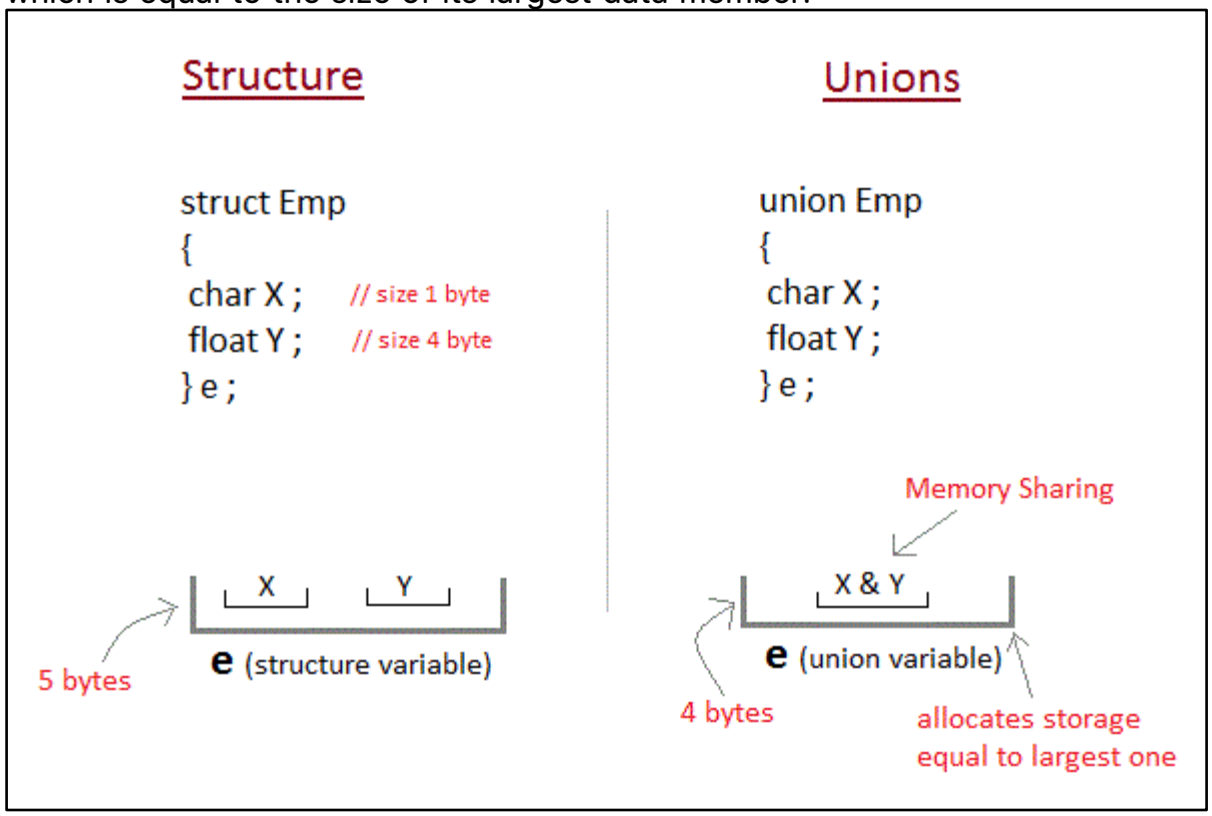
```
    printf("%f\n", it.b);
    printf("%c\n", it.ch);

    return 0;
}
```

Output
-26426
20.1999
 Z

Difference between union and structure

Unions are conceptually similar to **structures**. The syntax to declare/define a union is also similar to that of a structure. The only differences is in terms of storage. In **structure** each member has its own storage location, whereas all members of **union** uses a single shared memory location which is equal to the size of its largest data member.



Edit with WPS Office

The primary difference can be demonstrated by this example:

```
#include <stdio.h>
union unionJob
{
   //defining a union
   char name[32];
   float salary;
   int workerNo;
} uJob;

struct structJob
{
   char name[32];
   float salary;
   int workerNo;
} sJob;

int main()
{
   printf("size of union = %d", sizeof(uJob));
   printf("\nsize of structure = %d", sizeof(sJob));
   return 0;
}
```

Output

```
size of union = 32
size of structure = 40
```

More memory is allocated to structures than union

As seen in the above example, there is a difference in memory allocation between union and structure.

The amount of memory required to store a structure variable is the sum of memory size of all members.

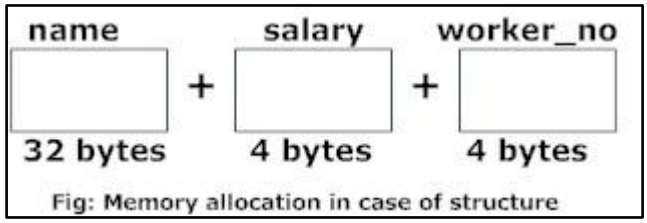


Fig: Memory allocation in case of structure

Edit with WPS Office

But, the memory required to store a union variable is the memory required for the largest element of an union.

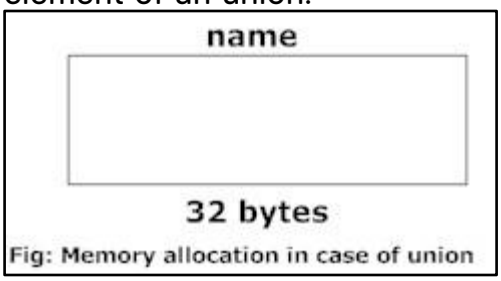


Fig: Memory allocation in case of union

Only one union member can be accessed at a time

In the case of structure, all of its members can be accessed at any time.
But, in the case of union, only one of its members can be accessed at a time and all other members will contain garbage values

```
#include <stdio.h>
union job
{
   char name[32];
   float salary;
   int workerNo;
} job1;

int main()
{
   printf("Enter name:\n");
   scanf("%s", &job1.name);

   printf("Enter salary: \n");
   scanf("%f", &job1.salary);

   printf("Displaying\nName :%s\n", job1.name);
   printf("Salary: %.1f", job1.salary);

   return 0;
}
```

Output
Enter name

Edit with WPS Office

Hillary
Enter salary
1234.23
Displaying
Name: f%Bary
Salary: 1234.2

Note: You may get different garbage value or empty string for the name.
Initially in the program, *Hillary* is stored in job1.name and all other members of *job1*, i.e. salary, workerNo, will contain garbage values.
But, when user enters the value of salary, 1234.23 will be stored in job1.salary and other members, i.e. name, workerNo, will now contain garbage values.
Thus in the output, *salary* is printed accurately but, *name* displays some random string.

34. Write a program to read and print student details using structure pointers

```
#include <stdio.h>
#include <stdlib.h> // Required for malloc and free

// Define a structure to hold student details
struct Student {
    char name[50];
```

Edit with WPS Office

```
    int roll_no;
    float marks;
};

// Function to read student details
void readStudentDetails(struct Student *student) {
    printf("Enter student's name: ");
    scanf("%s", student->name);
    printf("Enter roll number: ");
    scanf("%d", &student->roll_no);
    printf("Enter marks: ");
    scanf("%f", &student->marks);
}

// Function to print student details
void printStudentDetails(const struct Student *student) {
    printf("\nStudent Details:\n");
    printf("Name: %s\n", student->name);
    printf("Roll Number: %d\n", student->roll_no);
    printf("Marks: %.2f\n", student->marks);
}

int main() {
    // Allocate memory for a student
    struct Student *studentPtr = (struct Student *)malloc(sizeof(struct Student));
    if (studentPtr == NULL) {
        printf("Memory allocation failed!\n");
        return 1; // Exit if memory allocation fails
    }

    // Read student details
    readStudentDetails(studentPtr);

    // Print student details
    printStudentDetails(studentPtr);

    // Free allocated memory
    free(studentPtr);

    return 0;
}

Enter student's name: John
Enter roll number: 101
Enter marks: 85.5
```

Edit with WPS Office

Student Details:
Name: John
Roll Number: 101
Marks: 85.50

35. Write a program to store Information and Display it using

```
#include <stdio.h>
#include <stdlib.h> // Required for malloc and free

// Define a structure to hold student details
struct Student {
    char name[50];
    int roll_no;
    float marks;
};

// Function to read student details
void readStudentDetails(struct Student *student) {
    printf("Enter student's name: ");
    scanf("%s", student->name);
    printf("Enter roll number: ");
    scanf("%d", &student->roll_no);
```


Edit with WPS Office

```
    printf("Enter marks: ");
    scanf("%f", &student->marks);
}

// Function to print student details
void printStudentDetails(const struct Student *student) {
    printf("\nStudent Details:\n");
    printf("Name: %s\n", student->name);
    printf("Roll Number: %d\n", student->roll_no);
    printf("Marks: %.2f\n", student->marks);
}

int main() {
    int numStudents;

    // Ask user for the number of students
    printf("Enter the number of students: ");
    scanf("%d", &numStudents);

    // Allocate memory for an array of students
    struct Student *students = (struct Student *)malloc(numStudents * sizeof(struct
Student));
    if (students == NULL) {
        printf("Memory allocation failed!\n");
        return 1; // Exit if memory allocation fails
    }

    // Read details for each student
    for (int i = 0; i < numStudents; i++) {
        printf("\nEnter details for student %d:\n", i + 1);
        readStudentDetails(&students[i]);
    }

    // Print details for each student
    for (int i = 0; i < numStudents; i++) {
        printStudentDetails(&students[i]);
    }

    // Free allocated memory
    free(students);

    return 0;
}

Enter the number of students: 2
```

Edit with WPS Office

```
Enter details for student 1:
Enter student's name: Alice
Enter roll number: 101
Enter marks: 90.5

Enter details for student 2:
Enter student's name: Bob
Enter roll number: 102
Enter marks: 85.0

Student Details:
Name: Alice
Roll Number: 101
Marks: 90.50

Student Details:
Name: Bob
Roll Number: 102
Marks: 85.00
```


Edit with WPS Office

36. Write a program to add two numbers using structure to a function Structure.

```
#include <stdio.h>

// Define a structure to hold two numbers
struct Numbers {
    float num1;
    float num2;
};

// Function to add two numbers using a structure
float addNumbers(struct Numbers numbers) {
    return numbers.num1 + numbers.num2;
}

int main() {
    // Declare a variable of type Numbers
    struct Numbers numbers;

    // Input two numbers
    printf("Enter first number: ");
    scanf("%f", &numbers.num1);
    printf("Enter second number: ");
    scanf("%f", &numbers.num2);

    // Call the function to add the numbers
    float sum = addNumbers(numbers);

    // Display the result
    printf("The sum of %.2f and %.2f is %.2f\n", numbers.num1, numbers.num2, sum);

    return 0;
}
```

```
Enter first number: 5.5
Enter second number: 4.5
The sum of 5.50 and 4.50 is 10.00
```


Edit with WPS Office

37. **Write a program to create a array of structure for five students.**

```
#include <stdio.h>

// Define a structure to hold student details
struct Student {
    char name[50];
    int roll_no;
    float marks;
};

int main() {
    // Declare an array of structures for five students
    struct Student students[5];

    // Input details for each student
    for (int i = 0; i < 5; i++) {
        printf("\nEnter details for student %d:\n", i + 1);
        printf("Enter student's name: ");
        scanf("%s", students[i].name);
        printf("Enter roll number: ");
        scanf("%d", &students[i].roll_no);
        printf("Enter marks: ");
        scanf("%f", &students[i].marks);
    }

    // Print details for each student
    printf("\nStudent Details:\n");
    for (int i = 0; i < 5; i++) {
        printf("\nStudent %d:\n", i + 1);
        printf("Name: %s\n", students[i].name);
        printf("Roll Number: %d\n", students[i].roll_no);
        printf("Marks: %.2f\n", students[i].marks);
    }

    return 0;
}
```

```
Enter details for student 1:
Enter student's name: Alice
Enter roll number: 101
Enter marks: 90.5

Enter details for student 2:
Enter student's name: Bob
```


Edit with WPS Office

```
Enter roll number: 102
Enter marks: 85.0

Enter details for student 3:
Enter student's name: Charlie
Enter roll number: 103
Enter marks: 78.5

Enter details for student 4:
Enter student's name: Diana
Enter roll number: 104
Enter marks: 92.0

Enter details for student 5:
Enter student's name: Ethan
Enter roll number: 105
Enter marks: 88.5

Student Details:

Student 1:
Name: Alice
Roll Number: 101
Marks: 90.50

Student 2:
Name: Bob
Roll Number: 102
Marks: 85.00

Student 3:
Name: Charlie
Roll Number: 103
Marks: 78.50

Student 4:
Name: Diana
Roll Number: 104
Marks: 92.00

Student 5:
Name: Ethan
Roll Number: 105
Marks: 88.50
```


Edit with WPS Office

Lab Assignment-11 (File Handling)
File creation, types of file, modes of file (r, w, a, r+, w+, a+), different operation on file reading, writing, and appending, reading and writing structure data from/to the file.

File Handling in C
In C programming, file is a place on your physical disk where information is stored.

Why files are needed?

- When a program is terminated, the entire data is lost. Storing in a file will preserve your data even if the program terminates.
- If you have to enter a large number of data, it will take a lot of time to enter them all. However, if you have a file containing all the data, you can easily access the contents of the file using few commands in C.
- You can easily move your data from one computer to another without any changes.


Edit with WPS Office

Types of Files

When dealing with files, there are two types of files you should know about:

1. Text files
2. Binary files

1. Text files

Text files are the normal .txt files that you can easily create using Notepad or any simple text editors.
When you open those files, you'll see all the contents within the file as plain text. You can easily edit or delete the contents.
They take minimum effort to maintain, are easily readable, and provide least security and takes bigger storage space.

2. Binary files

Binary files are mostly the .bin files in your computer.
Instead of storing data in plain text, they store it in the binary form (0's and 1's).
They can hold higher amount of data, are not readable easily and provides a better security than text files

File Operations

In C, you can perform four major operations on the file, either text or binary:

1. Creating a new file
2. Opening an existing file
3. Closing a file
4. Reading from and writing information to a file

Working with files

When working with files, you need to declare a pointer of type file. This declaration is needed for communication between the file and program.
FILE *fptr;

Opening a file - for creation and edit

Opening a file is performed using the library function in the **"stdio.h"** header file: fopen().
The syntax for opening a file in standard I/O is
ptr = fopen("fileopen","mode")
For Example:


Edit with WPS Office

fopen("E:||cprogram||newprogram.txt","w");

fopen("E:||cprogram||oldprogram.bin","rb");

Explanation

- Let's suppose the file newprogram.txt doesn't exist in the location E:\cprogram. The first function creates a new file named newprogram.txt and opens it for writing as per the mode'w'.
  The writing mode allows you to create and edit (overwrite) the contents of the file.
- Now let's suppose the second binary file oldprogram.bin exists in the location E:\cprogram. The second function opens the existing file for reading in binary mode 'rb'.
  The reading mode only allows you to read the file, you cannot write into the file

Opening Modes in Standard I/O

File Mode	Meaning of Mode	During Inexistence of file
r	Open for reading.	If the file does not exist, fopen() returns NULL.
rb	Open for reading in binary mode.	If the file does not exist, fopen() returns NULL.
w	Open for writing.	If the file exists, its contents are overwritten. If the file does not exist, it will be created.
wb	Open for writing in binary mode.	If the file exists, its contents are overwritten. If the file does not exist, it will be created.
a	Open for append. i.e, Data is added to end of file.	If the file does not exists, it will be created.
ab	Open for append in binary mode. i.e, Data is added to end of file.	If the file does not exists, it will be created.
r+	Open for both reading and writing.	If the file does not exist, fopen() returns NULL.
rb+	Open for both reading	If the file does not

Edit with WPS Office

	and writing in binary mode.	exist, fopen() returns NULL.
w+	Open for both reading and writing.	If the file exists, its contents are overwritten. If the file does not exist, it will be created.
wb+	Open for both reading and writing in binary mode.	If the file exists, its contents are overwritten. If the file does not exist, it will be created.
a+	Open for both reading and appending.	If the file does not exists, it will be created.
ab+	Open for both reading and appending in binary mode.	If the file does not exists, it will be created.

Closing a File

The file (both text and binary) should be closed after reading/writing.
Closing a file is performed using library function *fclose().*
fclose(fptr); //fptr is the file pointer associated with file to be closed

Reading and writing to a text file

For reading and writing to a text file, we use the functions fprintf() and fscanf().
They are just the file versions of printf() and scanf(). The only difference is that, fprint and fscanf expects a pointer to the structure FILE.

Writing to a text file

Example 1: Write to a text file using fprintf()

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
   int num;
   FILE *fptr;
   fptr = fopen("C:\\program.txt","w");
```

Edit with WPS Office

```
   if(fptr == NULL)
   {
      printf("Error!");
      exit(1);
   }
   printf("Enter num: ");
   scanf("%d",&num);
   fprintf(fptr,"%d",num);
   fclose(fptr);
   return 0;
}
```

This program takes a number from user and stores in the file program.txt. After you compile and run this program, you can see a text file program.txt created in C drive of your computer. When you open the file, you can see the integer you entered.

Reading from a text file

Example 2: Read from a text file using fscanf()

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
   int num;
   FILE *fptr;
   if ((fptr = fopen("C:\\program.txt","r")) == NULL){
       printf("Error! opening file");

       // Program exits if the file pointer returns NULL.
       exit(1);
   }
   fscanf(fptr,"%d", &num);
   printf("Value of n=%d", num);
   fclose(fptr);
   return 0;
}
```

This program reads the integer present in the program.txt file and prints it onto the screen. If you successfully created the file from **Example 1**, running this program will get you the integer you entered.


Edit with WPS Office

Reading and writing to a binary file

Functions fread() and fwrite() are used for reading from and writing to a file on the disk respectively in case of binary files.

Writing to a binary file

To write into a binary file, you need to use the function fwrite(). The functions takes four arguments: Address of data to be written in disk, Size of data to be written in disk, number of such type of data and pointer to the file where you want to write.
fwrite(address_data,size_data,numbers_data,pointer_to_file);

Example 3: Writing to a binary file using fwrite()

```
#include <stdio.h>
#include <stdlib.h>
struct threeNum
{
   int n1, n2, n3;
};

int main()
{
   int n;
   struct threeNum num;
   FILE *fptr;
   if ((fptr = fopen("C:\\program.bin","wb")) == NULL){
       printf("Error! opening file");
       exit(1);             // Program exits if the file pointer returns NULL.

   }
   for(n = 1; n < 5; ++n)
   {
      num.n1 = n;
      num.n2 = 5*n;
```


Edit with WPS Office

```
    num.n3 = 5*n + 1;
    fwrite(&num, sizeof(struct threeNum), 1, fptr);
  }
  fclose(fptr);
  return 0;
}
```

In this program, you create a new file program.bin in the C drive.
We declare a structure threeNum with three numbers - *n1, n2 and n3*, and define it in the main function as num.
Now, inside the for loop, we store the value into the file using fwrite.
The first parameter takes the address of *num* and the second parameter takes the size of the structure threeNum. Since, we're only inserting one instance of *num*, the third parameter is 1. And, the last parameter *fptr points to the file we're storing the data.

Reading from a binary file

Function fread() also take 4 arguments similar to fwrite() function as above
fread(address_data,size_data,numbers_data,pointer_to_file);

Example 4: Reading from a binary file using fread()

```
#include <stdio.h>
#include <stdlib.h>
struct threeNum
{
  int n1, n2, n3;
};
int main()
{
  int n;
  struct threeNum num;
  FILE *fptr;
  if ((fptr = fopen("C:\\program.bin","rb")) == NULL){
      printf("Error! opening file");
      exit(1);         // Program exits if the file pointer returns NULL.
  }
  for(n = 1; n < 5; ++n)
  {
    fread(&num, sizeof(struct threeNum), 1, fptr);
    printf("n1: %d\tn2: %d\tn3: %d", num.n1, num.n2, num.n3);
  }
  fclose(fptr);
  return 0;
}
```


Edit with WPS Office

in this program, you read the same file program.bin and loop through the records one by one.In simple terms, you read one threeNum record of threeNum size from the file pointed by *fptr into the structure *num*.

38. **Write a program to read text from a file.**

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *file;  // File pointer
    char filename[100];
    char ch;

    // Get the filename from the user
    printf("Enter the filename to read from: ");
    scanf("%s", filename);

    // Open the file in read mode
    file = fopen(filename, "r");

    // Check if the file was opened successfully
    if (file == NULL) {
        printf("Could not open file %s\n", filename);
        return 1; // Exit if file opening fails
    }

    // Read and display the content of the file
    printf("Contents of the file %s:\n", filename);
    while ((ch = fgetc(file)) != EOF) {  // Read one character at a time
        putchar(ch);  // Print the character to the console
    }

    // Close the file
    fclose(file);

    return 0;
}
```

```
Hello, this is a test file.
It contains multiple lines of text.
Reading from files in C is simple!

Enter the filename to read from: sample.txt
Contents of the file sample.txt:
Hello, this is a test file.
```


Edit with WPS Office

It contains multiple lines of text.
Reading from files in C is simple!

39. Write a program to write sentence in a file.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *file;  // File pointer
    char filename[100];
    char sentence[256];

    // Get the filename from the user
    printf("Enter the filename to write to: ");
    scanf("%s", filename);

    // Open the file in write mode
    file = fopen(filename, "w");

    // Check if the file was opened successfully
    if (file == NULL) {
        printf("Could not open file %s for writing\n", filename);
        return 1; // Exit if file opening fails
    }

    // Get the sentence to write to the file
    printf("Enter a sentence to write to the file: ");
    getchar(); // To consume the newline character left by scanf
    fgets(sentence, sizeof(sentence), stdin); // Read a sentence including spaces

    // Write the sentence to the file
    fprintf(file, "%s", sentence);

    // Close the file
    fclose(file);

    printf("Successfully written to %s\n", filename);

    return 0;
}
```

Enter the filename to write to: output.txt
Enter a sentence to write to the file: This is a sample sentence written to the file.
Successfully written to output.txt

Edit with WPS Office

40. Write a program to find the total number of lines in a file.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *file;  // File pointer
    char filename[100];
    char ch;
    int line_count = 0;

    // Get the filename from the user
    printf("Enter the filename to count lines: ");
    scanf("%s", filename);

    // Open the file in read mode
    file = fopen(filename, "r");

    // Check if the file was opened successfully
    if (file == NULL) {
        printf("Could not open file %s\n", filename);
        return 1; // Exit if file opening fails
    }

    // Read characters from the file and count lines
    while ((ch = fgetc(file)) != EOF) {  // Read one character at a time
        if (ch == '\n') {  // Check for newline character
            line_count++;
        }
    }

    // Close the file
    fclose(file);

    // If the file is not empty, increment the line count for the last line
    if (line_count > 0 || (line_count == 0 && ch != EOF)) {
        line_count++;
    }

    // Print the total number of lines
    printf("Total number of lines in the file %s: %d\n", filename, line_count);

    return 0;
}
```

Edit with WPS Office

Hello, this is line 1.
This is line 2.
And here is line 3.

Enter the filename to count lines: sample.txt
Total number of lines in the file sample.txt: 3

Edit with WPS Office